

# 03. Vim 从入门到得体

## 目录

- 03. Vim 从入门到得体
  - 目录
  - 前言
  - 第一部分：守夜者的十节课（必修）
    - 第一课：从“山寨货”到“服务器标配”：Vim 的传奇起点
    - 第二课：打开、编辑与保存 —— 活着进去，活着出来
    - 第三课：移动光标 —— 让手指告别方向键
    - 第四课：插入模式 —— 不只是按个 `i` 那么简单
    - 第五课：`.` 命令 —— 做一次，重复无数次
    - 第六课：删除、撤销与恢复 —— 做错了也不怕
    - 第七课：复制、粘贴与缩进 —— 搬砖的艺术
    - 第八课：文件信息、跳转、定位括号和缩进 —— 别迷路
    - 第九课：搜索、替换 —— 在文件中“抓人”
    - 第十课：Vim 的哲学 —— 你站在了能看见风景的地方
  - 第二部分：通往专家之路（选修）
    - 我应该从哪个选修开始？
    - 选修1：文本对象 —— 操作的单位升级了
    - 选修2：寄存器 —— Vim 的“多个剪贴板”
    - 选修3：宏 —— 录制操作，一键重放
    - 选修4：Vim 是一门语言 —— 命令组合的逻辑
    - 选修5：配置 `.vimrc` —— 让 Vim 变成你的形状
    - 选修6：插件生态 —— 把 Vim 变成 IDE
    - 选修7：多文件与多窗口 —— 告别“一个文件一个终端”
  - 附录
    - 附录 A：vi 与 vim 的完整对比表
    - 附录 B：常用命令速查卡
    - 附录 C：推荐阅读 —— 《Vim 实用技巧（第 2 版）》
    - 附录 D：常见坑与解法（新手救急）
  - 写在最后

---

## 前言

### 这本教程是给谁看的？

你是不是也觉得，Vim 是那种“人人都说好，但人人都没用明白”的东西？

**你身边一定有这样的朋友：**兴致勃勃装上 Vim，练了两天就放弃了，丢下一句“反人类”。或者在服务器上不小心进了 Vim，死活退不出来，最后只能关掉终端重开——然后假装什么都没发生过。又或者搜了一堆教程，学完只会 `i` 和 `:wq`，用起来跟记事本没区别，感觉 Vim 吹得再好，好像跟自己没什么关系。

你可能就是那个朋友。也可能你正在成为那个朋友的路上——然后你翻开了这本教程。

**这本教程不追求“精通”，只追求“得体”。**

什么叫得体？

- 打开 Vim 不慌，知道自己现在在哪个模式、该按什么键
- 手指习惯性地放在主键盘区，不再低头找方向键
- 写代码的时候不会觉得“Vim 在拖我后腿”——它可能还没帮你飞，但至少不拽你了

**目标就三个字：不难受。**

## 这本教程不是什么？

**不是《Vim 实用技巧》的替代品。** 那本书是真正的“绝世武功”，但它的符号表示法（`<C-x>`、`>G` 之类的）会把零基础读者直接劝退。这本教程是你练功之前的热身操——够你用了，剩下的你自己决定要不要继续走。

**不是命令大全。** 我只讲 **你 80% 的时间里会用到的 20% 的命令**。你不需要背完所有指令才能开始用。那剩下的 80% 你一辈子可能都用不上。

**不是插件配置教程。** 在学会基础操作之前，别碰插件。插件是给懂 Vim 的人用的，不是给正在学的人用的。现在别分心。

## 怎么读？

第一课（安装 & 历史）

    | 先把 Vim 请进门，知道它从哪来

    ↓

第二课（打开/保存）

    | 第一步先学会“活着回来”——打开、编辑、保存、退出

    ↓

第三课（移动光标）

    | 手指离开方向键，用 `hjkl` 在屏幕里走路

    ↓

第四课（模式切换）

    | Vim 最大的秘密——它为什么跟所有编辑器都不一样

    ↓

第五课（`.` 命令）

    | Vim 的精髓，所有高手的肌肉记忆

    ↓

第六、七、八、九课（编辑命令）

    | 给 `.` 命令提供“弹药”——你学会删除、撤销、搜索、替换之后，`.` 才有威力

    ↓

第十课（哲学总结）

    | 你站在了能看见风景的地方

每天只学 1-2 课，每课学完练 10 分钟。不是“建议练”，是“必须练”。光看不练，等于没看。

## 给一个承诺

读完前十课，你能做到三件事：

1. 打开 Vim、编辑内容、保存退出——全套流程顺手，不再卡壳
2. 光标移动、删除、撤销、搜索、替换——手指有了记忆，不再用方向键
3. 不再觉得“Vim 是反人类的”——你至少能理解了为什么有人愿意用它

**如果十节课之后你觉得自己什么都没学会，那只有一个可能：你跳过了练习。**

Vim 的学习曲线陡得像悬崖。这本教程是在悬崖上给你凿出来的第一级台阶。十节课之后，你会站在一个能看见风景的地方——不是山顶，但你已经不是在爬了，你是在走。

守夜者学 Vim，不是为了炫技。是为了在只有命令行的世界里，还能写、还能改、还能守住。

今夜如此，夜夜皆然。

## 第一部分：守夜者的十节课（必修）

**目标：**让你能“得体”地用 Vim 写代码、改配置、活着退出来。

**读完这一部分，你已经超越了 80% 的 Vim 初学者。**

## 第一课：从“山寨货”到“服务器标配”：Vim 的传奇起点

### 1.1 Vim 是什么？

Vim 是一个在终端里运行的文本编辑器。

和你在《玩转 VS Code》里用过的编辑器比，它少了很多东西：

- **没有** 鼠标
- **没有** 菜单栏
- **没有** 可以点的图标
- **没有** 任何“图形界面”的东西

你在屏幕上看到的，就是一行行文字，和一个闪烁的光标。

听起来像“反人类”对吧？但 Vim 有一个核心逻辑：**你的手指永远不用离开键盘。**

所有操作都在主键盘区完成——你不用抬手去摸鼠标，不用去点菜单栏，不用等它加载什么动画。习惯了之后，你写东西的速度会比用鼠标快。

但快不是重点。重点是：

**Vim 不需要安装。**

你在《玩转 Markdown》里学会了用笔记记录思考，在《玩转 VS Code》里学会了用编辑器高效写作。然后有一天，你 SSH 进了一台服务器——没有桌面，没有浏览器，没有 VS Code。你只有一个命令行窗口。你怎么改配置文件？你怎么写脚本？你怎么在服务器上活下来？

答案是：Vim。

*Kate 问：“所以我不需要装它？”*

“对。绝大多数 Linux 服务器上，Vim 已经在那里等你了。不是你选择了它，是环境里只有它。”

“.....所以我不是因为它好用才学的。我是因为没得选？”

“对。但学完之后你会发现——你开始主动选它了。”

---

## 1.2 一个“山寨货”的逆袭

1976 年，一个叫 Bill Joy 的程序员写了一个编辑器，叫 **Vi**。

Vi 很强大，但它不是免费的。你要用，得付钱。

1988 年，一个叫 **Bram Moolenaar** 的荷兰程序员遇到了一个问题：他用不了 Vi，因为太贵了。于是他做了一个决定——既然用不起，那我自己写一个“高仿版”。

他给这个项目起名叫 **Vi IMitation**，缩写就是 **Vim**——直译过来就是“Vi 的山寨版”。

后来这个“山寨货”越做越强，功能超过了原版。名字也改了，从“模仿品”改成了 **Vi IMproved**（Vi 的改良版）。

现在，原版 Vi 已经很少人用了，Vim 成了几乎所有 Linux 发行版的默认安装项。你打开终端，输入 `vim`，它就出来了。

**所以别怕从模仿开始。Vim 自己就是从模仿开始的。**

*Kate 问：“所以一个‘高仿版’，最后把原版挤掉了？”*

“对。因为它是免费的，而且做得比原版好用。这个故事告诉我们：别看不起‘山寨’——当它做得足够好，它就是‘改良版’。”

---

## 1.3 为什么用 `hjkl` 移动光标？

Vim 里用 `h` `j` `k` `l` 代替方向键。

你可能会问：“为什么不用方向键？”

答案是：**1976 年的键盘上，方向键不在你现在的位置。**

看这张图。这是 Bill Joy 当年用的机器（ADM-3A）的键盘：



仔细看 **H**、**J**、**K**、**L** 这四个键——上面印着方向箭头。

当时的独立方向键在键盘右边，离主键盘区很远。每移动一次光标，手都要整个挪过去。Bill Joy 觉得这样太慢了，就直接用 **H**、**J**、**K**、**L** 当方向键——这样手指就不用离开主键盘区。

他把这个习惯写进了 Vi 的代码里。Vim 继承了下来，一直传到今天。

### 现在，做一件事：

把右手食指放在 **J** 键上。摸到了吗？那个键上有一个小凸起——那是所有键盘上用来帮你定位的“锚点”。食指找到它之后，你的手指会自然覆盖在 **H**、**J**、**K**、**L** 四个键上：

- 向右 → **L**
- 向左 → **H**
- 向下 → **J**
- 向上 → **K**

这就是 Vim 的核心移动区，也是你接下来几节课要反复练习的“家”。

Kate 摸了摸 J 键上的小凸起，说：“所以这不是乱安排的。是手指放上去，刚好能碰到这四个键。”

“对。最舒服的姿势，就是最高效的姿势。”

“那我现在应该开始练了？”

“对。把你面前这段文字里的光标，用 `hjk1` 从头走到尾。每个方向都走几次。不用快——但你得让你的手指记住它们的位置。”

## 1.4 什么是终端？

Vim 运行在 **终端** 里。

终端就是那个“黑框框”。你在电影里看到黑客敲代码的地方，就是它。

不同系统叫法不同：

- **Windows**：叫“命令提示符”或“PowerShell”
- **Mac**：叫“终端”
- **Linux**：叫“终端”

如果你没用过终端，现在打开一个看一眼。别怕，它就是个黑框框。你以后要在里面打很多字。

Kate 打开终端盯着看了几秒：“它就一个光标闪啊闪，等我跟它说话？”

“对。你输入 `vim`，它就给你 Vim。你输入别的东西，它也会听话。只是别乱输入你不认识的命令就行。”

**小结**：终端是 Vim 的家。先打开终端，再启动 Vim。

---

## 1.5 安装 Vim

不知道你的电脑是什么系统？

- 左下角有“开始”菜单 → Windows
- 左上角有苹果图标 → Mac
- 不知道 → 大概率是 Windows

选你对应的系统，往下看。

**注意**：Vim 原生的界面很丑，如果你看不惯或不喜欢的话可以直接去看[选修5：配置 .vimrc](#) —— [让 Vim 变成你的形状](#)，放心好了，这节是教你如何 **配置 Vim**，跟 **如何用 Vim** 不是一回事，不需要前几课的基础。

---

### 1.5.1 Windows 用户

Windows 没有 Vim。你得先装一个能让它跑起来的东西——**WSL**（Windows 上的 Linux 子系统）。

别被名字吓到。WSL 就是在你 Windows 电脑里“装了一台小电脑”，那台小电脑跑的是 Linux，Vim 就住在里面。装一次，以后不用再管它。

Kate 举手：“等等，我一定要用命令行装吗？我看到‘以管理员身份运行’就紧张。”

“不用怕，Kate。你也可以去下载安装包，双击安装，像装普通软件一样。我两个方法都写，你选一个看得顺眼的。”

**方法一：一行命令搞定（推荐，最快，但会安装在 C 盘）**

1. 打开“开始”菜单，搜 `PowerShell`
2. 右键点“PowerShell”，选“以管理员身份运行”
3. 在黑框框里粘贴这行命令，按回车：

```
wsl --install
```

4. 等几分钟，然后**重启电脑**
5. 重启后，打开“开始”菜单，搜 `Ubuntu`，打开它
6. 第一次打开要等一会儿，然后让你设**用户名和密码**。随便写，别忘就行（Kate 写的是 `vimer`）
7. 进去后输入：

```
sudo apt update && sudo apt install vim -y
```

## 方法二：双击安装包（如果你不想碰命令行）

1. 打开浏览器，访问 <https://github.com/microsoft/WSL/releases>
2. 下载最新的 `wsl.2.x.x.x64.msi` 文件
3. 双击运行，一路“下一步”，装完**重启电脑**
4. 重启后打开开始菜单里的“Ubuntu”，创建用户名和密码
5. 进去后输入 `sudo apt update && sudo apt install vim -y`

### 想把 Ubuntu 装到 D 盘？

默认是 C 盘，如果你 C 盘快满了，可以换成 D 盘安装：

```
wsl --install -d Ubuntu --install-location D:\WSL
```

装之前敲这个命令就行。

## 1.5.2 Mac 用户

Mac 自带一个老版的 `vi`，但你要用新版 Vim。

1. 打开“启动台”→“其他”→“**终端**”
2. 先装 Homebrew（Mac 的软件管家）。在终端里粘贴这行命令：

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

等几分钟，中间可能让你按回车确认，就按

3. 装完后，输入：

```
brew install vim
```

## 1.5.3 Linux 用户

打开终端（按 `Ctrl+Alt+T` 最快），输入：

```
sudo apt install vim -y
```



如果报错说 `command not found`，把 `apt` 换成 `yum` 或 `pacman`——不同 Linux 的包管理器名字不一样。

---

## 1.6 验证安装

装完之后，在终端里输入：

```
vim --version
```

按回车。如果屏幕爆出一堆以 `Vim` 开头的文字，中间有 `Huge version` 之类的字眼——**成了**。

如果提示 `command not found`，说明没装上。不用急，关掉终端重新打开再输一次。不行的话，回看安装步骤，看哪一步敲错了。

---

好的。以下是修订后的 **1.7 第一次启动**，已将三种权限解决方案自然地融入对话和教学中。

---

## 1.7 第一次启动

在终端里输入 `vim`，按回车。

画面变了。屏幕上出现版本号、帮助提示，底下一行状态条。

**你现在在 Vim 的“普通模式”里。**

试试按一下字母 `i`——屏幕底部出现了 `-- INSERT --`，你能打字了。

现在别管模式是什么意思。你只需要知道一件事：

**你怎么出去？**

*Kate 按了 `i`，看到 `-- INSERT --` 出现了，说：“这步我会，但我怎么回去？”*

“好问题。这是今天最重要的操作，比你学到的所有东西都重要——因为你如果出不去，你就被困在 Vim 里了。”

**活着退出去的方法（救命四步）**

1. 按 `ESC`（键盘左上角）
2. 输入冒号 `:`（左下角会出现冒号）
3. 输入 `q!`（左下角变成 `:q!`）
4. 按 **回车**

屏幕一闪，你回到了终端。

`q!` 是什么意思？

- `:` 进入命令行模式



- q 就是 quit (退出)
- ! 是“别拦我”。你没保存内容，不加 ! Vim 会拦住你，加了 ! 就是告诉它：“我知道没保存，退！”

把这四个动作刻进脑子里：

```
ESC → : → q! → 回车
```

**这是你的救命稻草。你忘了任何操作都行，别忘这四个键。**

第二天，Kate 已经能熟练地进出 Vim 了。她甚至试着在系统配置文件里改了一行设置——然后发现自己没有保存权限。

“沉默者，我改完了，怎么存不上？Vim 提示 E45: 'readonly' option is set。”

“你改的是什么文件？”

“/etc/hosts。我想给 Vim 的备用地址加个本地解析。”

“那个文件只有超级用户能改。你有两个选择。”

“我改了半天了，不想退出重来。”

“那就用第二个。听好——这条命令会救你无数次。”

## 在 Vim 里获得写文件权限（实战技巧）

当你打开一个需要超级用户权限的文件，有三种解法。

### 解法一：打开之前就知道它需要权限（预防）

在终端里，用下面任意一条命令打开文件：

```
sudo vim 文件名
```

或者更安全的方式：

```
sudo -e 文件名
```

`sudo -e` 会在一个临时副本里编辑，保存时才写回原文件，不会让你的整个 Vim 进程以 root 权限运行。这是沉默者推荐的方式。

### 解法二：已经改了半天，才发现没权限（救命）

这是最让人血压飙升的场景——你已经改了十几处，准备 `:w` 保存，结果 Vim 冷冷地告诉你 `Permission denied`。

**不要退出。你的修改都还在。** 用这条命令：

```
:w !sudo tee %
```

回车后，Vim 会提示“文件已更改，是否重新加载？”，按 `L` 加载即可。

“这行命令做了什么？”Kate 盯着屏幕。

“它把你改的内容，用超级用户权限，写回当前文件。`%` 就是‘当前文件’的占位符。你不需要退出 Vim，不需要重新打开，不需要重改一遍。你只需要知道这条命令。”

**解法三：文件本身被设置了只读属性**

有时候文件可能被额外设置了只读属性，或者你不小心进入了只读模式。先试：

```
:set noreadonly
```

然后再 `:w` 保存。如果还是提示权限不足，退回解法二。

Kate 把这三条记在了她的作战手册上。在未来的内核开发中，她还会无数次用到 `:w !sudo tee %` ——但这是她第一次看见沉默者打字比她还快。

- `:w !sudo tee %` 是你必须刻进手指的肌肉记忆。你会在第三年写玩具内核时频繁遇到权限问题，这条命令每次都会救你。
- `sudo -e` 比 `sudo vim` 更安全——养成用它的习惯。
- 如果你连文件都还没开始改，就用解法一。如果你已经改了一堆东西才发现没权限，解法二永远在你手上。

**1.8 小结**

| 问题                           | 答案                                                            |
|------------------------------|---------------------------------------------------------------|
| Vim 是什么？                     | 终端里的文本编辑器，只能用键盘                                               |
| Vim 怎么来的？                    | 从“山寨货”逆袭成行业标杆                                                 |
| 为什么用 <code>hjkl</code> 移动光标？ | 40多年前那台机器的键帽上就是这么印的                                           |
| 终端是什么？                       | 一个黑框框，Vim 住在里面                                                |
| 怎么安装？                        | Windows 用 WSL，Mac 用 Homebrew，Linux 用 <code>apt</code>         |
| 怎么验证？                        | 输入 <code>vim --version</code> ，看到 Vim 就对                      |
| 怎么启动？                        | 终端里输入 <code>vim</code>                                        |
| 怎么退出去？                       | <code>ESC</code> → <code>:</code> → <code>q!</code> → 回车（背下来） |

## 1.9 本课作业

1. 搜一下：Bram Moolenaar（Vim 的作者）除了写 Vim，还做了什么让很多人佩服的事？
2. 打开终端，输入 `vim`，截一张启动界面的图存下来。这是你和 Vim 的“第一次见面”的照片。

**守夜者手记：**第一课的使命不是让你“会用”，是让你“不再怕”。你已经知道 Vim 从哪来、为什么用 `hjk1`、怎么装、怎么启动、怎么活着退出来——够了。

下一课，我们真的开始编辑文件了。打开、修改、保存、退出。也就是“活着进去，活着出来”。

## 第二课：打开、编辑与保存 —— 活着进去，活着出来

**这一课你将学会：**

- ✓ 打开一个文件（无论它存不存在）
- ✓ 在里面写字
- ✓ 把字存下来
- ✓ 安全地退出来

**完成后，你就能在 Vim 里完成一份真实笔记。** 不是“练习”，是真笔记。

### 2.1 打开文件：你第一次对 Vim 说“我要用你干活”

在终端里输入：

```
vim 笔记.md
```

按回车。

- 如果文件不存在 → Vim 帮你建一张空白页，等你写。
- 如果文件已存在 → Vim 打开它，把内容铺在屏幕上。

文件名可以带路径，比如 `vim ~/Documents/笔记.md`。

**这一瞬间，你在告诉 Vim：“我要干正事了。”**

### 2.2 写点东西：你第一次让 Vim 听你的话

屏幕亮了，光标在闪。

但你按键盘，什么都没出现。

**别慌。Vim 没坏。**

Vim 默认在“普通模式”——键盘上的字母不是用来打字的，是用来下命令的。

想打字？按 **i**。

```
i = insert = 我要开始打字了
```

按下去，屏幕底部会出现 `-- INSERT --`。现在你敲什么，它就出什么。

试试：

```
Hello, Vim!
```

你看，它听你的了。

*Kate 说：“我第一次打开 Vim，按了半天键盘没反应，我以为软件坏了。现在我知道了——我只是没按 i。”*

**你刚刚完成了你与 Vim 的第一次“对话”。** 你告诉它“我要打字”，它让你打了。

---

## 2.3 保存：你第一次让 Vim 记住你的话

你打了字。但这时候如果直接关掉终端，字就没了。

你得把它存下来。

按 **ESC**（回到普通模式），输入 `:w`，回车：

```
ESC → : → w → 回车
```

```
:w = write = 写入硬盘
```

左下角出现 `"笔记.md" [新] 已写入`——你的第一句话，被 Vim 记住了。

**你刚刚完成了你与 Vim 的第一次“合作”。** 你写，它存，不丢。

---

## 2.4 保存并退出：你的日常闭环

你写完了，想关掉文件走人：

```
ESC → : → wq → 回车
```

```
:wq = write + quit = 存完就走
```

怎么记住它？

想一想 `:wq` 的读音——像不像“我去”？写完存盘走人，确实就是“我去”了。

这是你以后每天做几十次的闭环动作。你完成了它，你就是“会用了”。

---

## 2.5 不保存退出：你的后悔药

万一写错了，或者改乱了，想直接走人，不存：

```
ESC → : → q! → 回车
```

```
:q! = quit + 强制 = 不存了，直接走
```

没有 `!`，Vim 会拦住你：“文件没保存，你真的要退吗？”

有 `!`，意思是：“别问了，我确定。”

*Kate 试了一次乱写一气，然后用 `:q!` 退出来，重新打开文件，发现乱码全没了，文件还是原来的样子。她说：“这个命令是我的后悔药。试错的时候特别好用——写了一大堆，觉得不对，直接 `:q!` 走人，不留痕迹。”*

你学会了 Vim 的撤销键。不是撤销一行，是撤销整次编辑。

---

## 2.6 完整流程：你已经在 Vim 里写完了一篇真实笔记

```
vim 守夜者笔记.md # 打开文件
```

按 `i` 进入插入模式

```
今天学了 Vim 的基本操作。
感觉还行，不算太难。
```

按 `ESC` 回到普通模式

```
:w 回车保存
```

按 `i` 继续插入

```
明天学移动光标。
```

按 `ESC` 回到普通模式

```
:wq 回车保存并退出
```

你完成了。你在 Vim 里写完了一份真实的、有两条内容的、保存在硬盘上的笔记。不是你练习时随手打的几行测试文字——是真的能留到明天的东西。

Kate 走完这套流程后说：“我可以在 Vim 里写笔记了。不是‘我会用Vim了’——是我真的拿它写了东西。这两段话明天还会在，我要记住这个感觉。”

## 2.7 本节课你能拿走的

打开文件 → 打字 → 存盘 → 继续打字 → 存盘走人  
vim 文件名 → i → :w → i → ESC → :wq

以及一个后悔药：ESC → :q!

我不用记住全部。你只需要记住这三样：

1. 按 i 才能打字
2. 按 ESC 才能下命令
3. :wq 存盘走人，:q! 不存走人

其他全是细枝末节。

## 2.8 本课作业：做一件真实的事

1. 用 Vim 新建一个文件，叫 vim-测试.md
2. 在里面写一句话：“今天是 \_\_\_\_ 年 \_\_\_\_ 月 \_\_\_\_ 日，我开始学 Vim。”
3. 保存并退出
4. 重新打开它，确认内容还在
5. 追加一句话，然后**不保存退出** (:q!)
6. 重新打开，确认第二句话**没有被保存**

**完成后你确定了一件事：**你打开、编辑、保存、不保存退出——每种操作你都亲手做了一遍。你不是“看过”，你是“做过”。

## 第三课：移动光标 —— 让手指告别方向键

这一课你将学会：

- ✓ 用 hjkl 代替方向键
- ✓ 在文件里快速跳转：行首、行尾、文件首、文件尾
- ✓ 按词移动，不再一个一个字符爬

**完成后，你移动光标的速度会快一倍。**不是“快了”，是“你不再需要低头找方向键了”。

### 3.1 核心四键：你的手指有了新家

上一课你学会了打字、保存、退出。你肯定用了方向键。

这一课，我们戒掉它。

**原因：** 方向键在键盘右边。每次按它，右手都得离开主键盘区。一天下来，你的右手在方向键和字母区之间来回跑——像上班通勤一样累。Vim的方案：用 `hjkl` 。

| 键              | 方向 | 怎么记住它     |
|----------------|----|-----------|
| <code>h</code> | 左  | 在左边，所以是左  |
| <code>j</code> | 下  | 像一支向下指的箭头 |
| <code>k</code> | 上  | 像一支向上指的箭头 |
| <code>l</code> | 右  | 在右边，所以是右  |

第一次用会觉得很别扭。所有 Vim 新手第一周都在跟这四个键打架。但一周之后，你的右手不会再想离开主键盘区——你会觉得“方向键在键盘右边”是个反人类的设计。

**你的“家”：**

把右手食指放在 `j` 键上。摸到那个小凸起了吗？

- 食指不动 → `j` → 向下
- 食指向上移一格 → `k` → 向上
- 食指不动、中指向左移 → `h` → 向左
- 食指不动、无名指向右移 → `l` → 向右

*Kate 把手放上去试了试，说：“我按了十年的方向键，现在你告诉我它们在那四个字母上。我的右手在抗议。”*

“抗议一周就好了。一周之后，你的右手会忘了方向键在哪。”

**马上做一件事：** 打开 Vim，打几行字，然后把手放在 `j` 上。用 `h j k l` 在行间上下移动。一开始会很慢，很别扭。但这是你“断奶”的第一步。

### 3.2 快速跳转：你不再需要一直按着 `j`

`hjkl` 适合“走两步”。如果你要从文件第一行跳到最后一行，一直按 `j` 按到手酸——Vim 有更快的办法：

| 命令              | 效果      | 怎么记住它                    |
|-----------------|---------|--------------------------|
| <code>0</code>  | 跳到当前行行首 | 行首啥也没有（ <code>0</code> ） |
| <code>\$</code> | 跳到当前行行尾 | 行尾有美金（ <code>%</code> ）  |
| <code>gg</code> | 跳到文件第一行 | “go go”，连按两次g            |



| 命令 | 效果                  | 怎么记住它   |
|----|---------------------|---------|
| G  | 跳到文件最后一行            | “Go”，大写 |
| ^  | 飞到行首 <b>第一个非空字符</b> | 跳过行首空格  |

gg 和 G 是你以后**最常用的两个移动命令**。没有之一。

试试：

- 打开一个长文件 → 按 gg → 光标飞到第一行
- 按 G → 光标飞到最后一行
- 按 0 → 回到当前行开头
- 按 \$ → 飞到当前行末尾

Kate 打开了她那篇200行的笔记，按了 gg，光标瞬间到了第一行。又按了 G，光标飞到最下面。她说：“我以前一直按着 j 不放，像个傻子。”

翻页：

| 命令     | 效果    | 怎么记住它        |
|--------|-------|--------------|
| Ctrl+f | 往下翻一页 | f = forward  |
| Ctrl+b | 往上翻一页 | b = backward |

### 3.3 按词移动：比“一个字一个字”快

h l 一个字一个字移太慢。Vim 可以按“词”跳：

| 命令 | 效果        | 怎么记住它        |
|----|-----------|--------------|
| w  | 跳到下一个单词开头 | w = word     |
| b  | 跳到上一个单词开头 | b = backward |
| e  | 跳到下一个单词末尾 | e = end      |

Kate 试了一下，说：“w 按一次跳一个词，l 按十次才到下一个词——这差太远了。”

### 3.4 练习：让手指记住，而不是让眼睛找

打开 Vim，复制这段文字：

Vim 是一个历史悠久的文本编辑器。

它诞生于 1988 年，最初叫 Vi IMitation。

后来改名为 Vi IMproved。

它的作者是 Bram Moolenaar。

Vim 有多种模式：普通模式、插入模式、可视模式、命令行模式。

按 `ESC` 回到普通模式，完成以下任务：

1. 用 `h j k l` 把光标移到“历史”这个词上
2. 按 `gg` 跳到第一行开头
3. 按 `G` 跳到最后一行末尾
4. 按 `0` 回到当前行开头
5. 按 `$` 飞到当前行末尾
6. 按 `w` 在单词之间跳几次

**目标：** 让手指记住这几个键的位置，而不是用眼睛去找方向键。

### 3.5 本节课你能拿走的

|                      |   |     |
|----------------------|---|-----|
| <code>h j k l</code> | → | 方向  |
| <code>gg</code>      | → | 到开头 |
| <code>G</code>       | → | 到末尾 |
| <code>0</code>       | → | 到行首 |
| <code>\$</code>      | → | 到行尾 |
| <code>w</code>       | → | 下一词 |
| <code>b</code>       | → | 上一词 |

**你最常使用的：**

- 移动一两行 → `j` 或 `k`
- 移到行首 → `0`
- 移到行尾 → `$`
- 跳到文件开头 → `gg`
- 跳到文件末尾 → `G`

**其他的，用到再说。** 你不用一次背完所有命令。先练熟这五个。

### 3.6 本课作业：断了方向键的奶

Kate 说：“我决定把键盘上的方向键抠掉。这样我的右手就再也不会去找它们了。”

“不用抠。你只需要把 `h j k l` 练到不用想就能按对。”

1. 用 Vim 新建一个文件，随便写点东西
2. 在普通模式里反复练习：

- `gg` 和 `G` 在首尾之间跳
- `0` 和 `$` 在行首尾之间跳
- `w` 和 `b` 在单词之间跳
- `h j k l` 在相邻字符之间微调

### 3. 全程不要碰方向键。

练到你能感觉“手指知道该按哪里，不用低头看键盘”。

**守夜者宣言：** 如果你这一课只学会一件事，那就是——**方向键在 Vim 的世界里不存在了。** 你的手指现在有了新家。继续走，它会越来越舒服。

下一课：插入模式的多种打开方式。

## 第四课：插入模式 —— 不只是按个 `i` 那么简单

这一课你将学会：

- ✓ 六种进入插入模式的方式，每种都有自己的“主场”
- ✓ 打完字立刻回家（`ESC`），不让手指在插入模式里迷路
- ✓ 把“模式切换”变成肌肉记忆

完成后，你不再是“在插入模式里待着”，而是“在普通模式里指挥，偶尔进入插入模式写几个字”。这是 Vim 高手和 Vim 新手的核心分界线。

### 4.1 六种方式：你不需要“背”，你需要“画面感”

先别看表格。先看这六张图：

|                |                             |
|----------------|-----------------------------|
| <code>i</code> | = 就在这写（insert）              |
| <code>a</code> | = 在后面接（after）               |
| <code>I</code> | = 行首开工（行首的 <code>i</code> ） |
| <code>A</code> | = 行尾补刀（行尾的 <code>a</code> ） |
| <code>o</code> | = 下面开张（open below）          |
| <code>O</code> | = 上面插队（open above）          |

把这六行读一遍，不要背，就当看弹幕。读到第三行，你会发现脑子里已经有画面了：

- `i` → “我就在光标这儿写字”
- `A` → “在行尾补个东西”——你写代码经常忘了加分号
- `o` → “在下面开一行接着写”

你真正需要记住的，其实只有这三个：`i`、`A`、`o`。

另外三个是“替身版”：`i` 的大写版 `I`（光标变行首），`a` 的大写版 `A`（光标后变行尾），`o` 的大写版 `O`（下面变上面）。小写变大写 = 范围放大。

你记住小写的三个，大写的自然能猜出来。

Kate 问：“那如果我写到一半，光标在行中间，但我想在行尾追加内容，用哪个？”

“A。直接按 A，不用先跳到行尾。它比 a 多了一个动作——先跳到行尾再进入插入模式。这就是为什么它叫‘强化版’。”

如果一定要背，用这套口令表：

| 命令 | 效果     | 口令（读一遍就记住） |
|----|--------|------------|
| i  | 光标前插入  | “就在这写”     |
| a  | 光标后插入  | “在后面接”     |
| I  | 行首插入   | “行首开工”     |
| A  | 行尾插入   | “行尾补一刀”    |
| o  | 下方新建一行 | “下面开张”     |
| O  | 上方新建一行 | “上面插队”     |

读两遍，你不会全记住。但 “行尾补一刀” 和 “下面开张” 会粘在你的脑子里——那就够了。

第三句实话：I 和 O 背不出来，是因为你还没用过它们。用一次就记住了。

## 4.2 写完就回家：ESC 是你新的回车键

你已经学会了 i 进入插入模式、打字、然后呢？

马上按 ESC 回到普通模式。

打字只是你的“外出散步”——打完就回来。你住在普通模式里，插入模式是你偶尔出去走两步的地方。

Kate 说：“每次打字都要按 ESC，好烦。”

“烦一周。一周之后你就感觉不到 ESC 了。到那时候，你在 VS Code 里打完字也会下意识去按左上角——然后发现 VS Code 不理你。你会笑一下，然后继续。”

如果你不按 ESC 会怎样：

- 你想用 j 移动光标 → 屏幕上出现了一个 j
- 你想用 dd 删一行 → 屏幕上出现了两个 d
- 你想用 h 向左移 → 屏幕上出现了一个 h

你以为 Vim 卡了，其实是你还在插入模式里，Vim 以为你在打字。

## 4.3 真正的高效：切换，而不是停留

Vim 的速度来自于“模式切换”，不是“在插入模式里打多快”。

普通模式（指挥） → 插入模式（写字几个字） → ESC（回家） → 普通模式（指挥）

这是 Vim 的呼吸节奏。写几个字，思考一下，移动一下，再写几个字。

**练习方法：**

1. 按 **i** → 打一个词 → **ESC**
2. 用 **h j k l** 移动几步 → **i** → 再打一个词 → **ESC**
3. 按 **A** → 在行尾打一个词 → **ESC**
4. 按 **o** → 在下一行打一个词 → **ESC**

一开始会觉得切换很烦。一周之后，你不会再“卡”在插入模式里。

## 4.4 本节课你能拿走的

常用的入口：**i**、**A**、**o**

唯一的出口：**ESC**

**Vim 的节奏就是：**写几个字，按 **ESC**，移动光标，再写几个字，再按 **ESC**。

## 4.5 本课作业：把“打字→ESC”练成肌肉记忆

1. 打开 Vim，新建一个文件
2. 打一行字，比如：

Vim 有六种进入插入模式的方式

3. 分别用 **i**、**a**、**I**、**A**、**o**、**O** 练习：
  - 在“Vim”前面加个“好”字（用 **i**）
  - 在“方式”后面加个“。”（用 **A**）
  - 在下面新建一行写“今天学到了”（用 **o**）
  - 在上面新建一行写“笔记：”（用 **O**）
4. 每做完一步，按 **ESC** 退出，然后想一下：刚才那一步有没有更快的进入方式？
5. 全程注意你的手指：你是不是打完字立刻按了 **ESC**？如果你是打完字愣了一会儿才按——说明你现在还在“想”而不是“本能”。

**守夜者宣言：**普通模式是家，插入模式是散步。写完就回家。这一课之后，你不会再在插入模式里按 **h j k l** ——因为你知道那是错的。

下一课：**.** 命令——Vim 的懒人键。

---

## 第五课：. 命令 —— 做一次，重复无数次

前四课你学会了移动光标、进入插入模式、保存退出。你现在已经能“用”Vim了——打开文件、写字、保存、关掉。

但“用”和“快”之间，隔着一条河。

这一课，我们过河。工具就是 . 命令。

Vim 的设计哲学里有一句话：“你做一遍，我帮你重复。” . 命令就是这句话的化身。所有高手的肌肉记忆，都建立在这一个键上。

---

### 5.1 什么是 . 命令？

. 是 Vim 里的一个命令：**重复上一次修改**。

“修改”的定义很宽——删除、插入、替换、缩进……几乎所有改变文本内容的操作，都算“修改”。

**简单说：**你做了一个改动，按 .，Vim 会原封不动地再做一次。

你看完这一整句，心里可能有句话：“就这？一个重复键？”

对，就一个键。但这一课的篇幅都在告诉你：**这一个键，是 Vim 最高频、最核心的命令。没有之一。**

---

### 5.2 第一次体验：你可能会愣住

打开 Vim，敲五行的文字：

```
Line 1  
Line 2  
Line 3  
Line 4  
Line 5
```

按 ESC 确保在普通模式。

1. 按 dd 删掉第 1 行（dd 是“删除整行”）
2. 按 . —— **第 2 行没了**
3. 再按 . —— **第 3 行没了**
4. 再按 . —— **第 4 行没了**

看着屏幕上的行一行行消失，你可能会愣住。这就是 . 命令给你的第一印象：**它记住了你刚做了什么，然后重复执行。**

Kate 试了一下，盯着屏幕说：“我按了三次点，它删了三行。而我什么都没数。”

---

“没错。你不需要数‘我要删 5 行’，你只需要说‘删一行’，然后按几次 `.`。脑子不用算数，手指不用重新敲 `dd`。”

### 5.3 . 命令能重复什么？

`.` 命令会记录你最近的一次普通模式下的编辑操作。它不记录移动光标、搜索等非编辑操作。

能重复的（这些都会）：

| 操作 | 命令示例                                               |
|----|----------------------------------------------------|
| 删除 | <code>dd</code> 、 <code>x</code> 、 <code>dw</code> |
| 修改 | <code>c</code> 、 <code>s</code> 、 <code>r</code>   |
| 插入 | <code>i</code> + 文字 + <code>ESC</code>             |
| 粘贴 | <code>p</code>                                     |
| 缩进 | <code>&gt;&gt;</code> 、 <code>&lt;&lt;</code>      |

不能重复的（这些不会覆盖 `.` 的记忆）：

- 移动光标：`h`、`j`、`k`、`l`、`w`、`b`、`gg`、`G`
- 搜索：`/`、`?`
- 撤销：`u`
- 保存：`:w`
- 退出：`:q`

移动光标的命令不会覆盖 `.` 的记忆。你做了修改 A，然后按了 100 次 `j`，`.` 依然重复修改 A——它只记“改了什么”，不记“中间走的路”。

### 5.4 “点公式”—— Vim 效率的核心公式

《Vim 实用技巧》这本书里，提出了一个概念叫“点公式”：

用一次按键移动光标，用另一次按键执行修改。  
移动 + 执行，再移动 + 再执行。

看两个例子你就懂了：

#### 例 1：给多行代码加分号

你有一组变量声明，每行末尾少了分号：

```
var foo = 1
var bar = 2
var baz = 3
```

第一次：

- 按 **A** → 跳到行尾，进入插入模式
- 输入 **;** → 加分号
- 按 **ESC** → 回到普通模式

然后：

- 按 **j** → 光标移到下一行
- 按 **.** → 重复刚才的“A;ESC”组合

你只需要：

```
A ; ESC → j . → j . → j .
```

**三行代码，一次手动，三次 **.**，全部搞定。**

Kate 试了加分号的例子，说：“我平时会一行一行地敲分号，或者用替换命令 `:s/$/;/`。你告诉我用 **j** 加 **.**？”

“两种都对。**.** 比替换命令更可控——你每按一次 **.**，都看得到效果。万一哪行不该加，你也不会改错。”

**例 2：在每行开头加 **#****

```
第一行  
第二行  
第三行
```

第一次：

- 按 **I** → 跳到行首，进入插入模式
- 输入 **#** → 加注释符
- 按 **ESC** → 回到普通模式

然后：

```
j . → j . → j .
```

**关键：****.** 记住了你做的**整个**修改，从进入插入模式到退出。**A**、**I**、**o**、**O** 这些命令，在 **.** 面前等价——它们都是“进入插入模式 → 打字 → 退出来”这一整段操作。

## 5.5 心态转变：从“我做 100 次”到“我做 1 次，Vim 做 99 次”

- 学 **.** 命令之前，你看到 100 行没有分号的代码，心里想的是“我要一行一行敲，敲到什么时候”。
- 学 **.** 命令之后，你看到 100 行没有分号的代码，心里想的是“做一次，然后按 99 次 **.**”。

工作量没变——但你的心态变了。从“我做 100 次”变成了“我做 1 次，Vim 帮我做 99 次”。



## 5.6 避坑提醒

`.` 只记录**最后一次修改**。如果你做了修改 A，然后做了修改 B（比如按 `dd` 删了一行），`.` 就变成重复修改 B 了。

如果你按了 `u` 撤销，`.` 不会变成“重复撤销”——它会重复你按 `u` 之前的那次修改。`u` 是撤销命令，不是“修改”命令，不会覆盖 `.` 的记忆。

## 5.7 本节课你能拿走的

`.` = 重复上一次修改

点公式：

移动 → `.` → 移动 → `.` → 移动 → `.` → .....

你不需要数“我要做多少次”。你只需要做一次，然后按 `.` 按到满意为止。

## 5.8 本课作业：删到没有行为止

1. 用 Vim 新建一个文件，输入 15 行文字
2. 按 `dd` 删掉第一行
3. 然后一直按 `.`，观察屏幕上的行一行一行消失
4. 不要数你按了多少次——按到文件空了为止
5. 输入另一个 15 行，每行前面加一个 `-`
  - 第一行：`I` → `-` → `ESC`
  - 后面：`j` → `.`（重复到完事）
6. 保存退出

**守夜者宣言：**删 15 行不厉害。但如果要删 150 行、1500 行呢？`.` 命令让你不用数“删几行”，只做“删一行”，然后重复。这一课的练习，就是让你感受这种“不数数”的快乐。

下一课：删除、撤销与恢复 —— 做错了也不怕。

## 第六课：删除、撤销与恢复 —— 做错了也不怕

上一课你学会了 `.` 命令——做一次修改，然后重复无数次。这一课我们回到“修改”本身：**怎么删、怎么改、怎么反悔。**

这一课你学会三件事：

- ☑ 删掉不想要的（比退格键快 10 倍）

- ✓ 改掉写错的（不用进入插入模式）
- ✓ 反悔做过的（删错了也不怕）

完成后，你修改文本的速度会比前五课快一个档次。

如果说普通模式下移动光标是“走路”，那么**删除、撤销、改错**就是“编辑三件套”——你在 Vim 里 80% 的编辑操作，都在这三种命令的范围内。这一课讲删除和撤销，后面几课会陆续解锁更高级的编辑命令。

### 6.1 删除基础命令

| 命令 | 效果         | 对应“动作”描述         |
|----|------------|------------------|
| x  | 删除光标下的一个字符 | “删掉光标这个位置上的一个字符” |
| X  | 删除光标前的一个字符 | “删掉光标前面的那个字符”    |
| dd | 删除整行       | “删掉这一整行”         |
| D  | 删除从光标到行尾   | “删掉从这里到行尾”       |

试试：

```
Hello World
```

光标停在 `w` 上：

- 按 `x` → 删掉 `w`，变成 `Hello orld`
- 按 `X` → 删掉 `o`（光标前的字符），变成 `Hell orld`（你试一下就明白）
- 按 `dd` → 整行消失
- 按 `D` → 从光标到行尾全部删除（光标在 `H` 上，按 `D` 整行清空，留下一个空行）

Kate 试了一下 `dd`，说：“之前删整行我要按着退格键好久。现在一下没了，感觉有点不真实。”

“这就是 Vim 的节奏。按一下，整个世界清净了。如果你删错了——先别慌，往下看。”

### 6.2 删除 + 移动 = 组合技

这是 Vim 真正的威力所在。

**公式：** `d` + 移动命令 = 删除从当前位置到目标位置之间的内容。

这个公式适用于 Vim 中所有带“动作”的命令。但这一课我们先只讲 `d` 的用法。

| 组合 | 效果          | 描述                                         |
|----|-------------|--------------------------------------------|
| dw | 删除到下一个单词的开头 | <code>d</code> + <code>w</code> （移动到下一个单词） |
| db | 删除到上一个单词的开头 | <code>d</code> + <code>b</code> （移动到上一个单词） |

| 组合  | 效果      | 描述               |
|-----|---------|------------------|
| d\$ | 删除到行尾   | d + \$ （移动到行尾）   |
| d0  | 删除到行首   | d + 0 （移动到行首）    |
| dG  | 删除到文件末尾 | d + G （移动到文件末尾）  |
| dgg | 删除到文件开头 | d + gg （移动到文件开头） |

这个公式的意思是：

“从光标这里开始，一路删到我想去的地方。”

- dw = “从这删到下一个词”
- d\$ = “从这删到行尾”
- dG = “从这删到文件末尾”

试试这一行，光标停在开头（H 上）：

```
Hello world from Vim
```

- 按 dw → 删掉 Hello （光标停在 world 前）
- 再按 dw → 删掉 world （光标停在 from 前）
- 再按 dw → 删掉 from （光标停在 Vim 前）

如果你想把整行删光，dd 更快。但 dw 适合“只删一部分，不改别的”。

Kate 试了 d\$，说：“我只想删掉这句话最后的几个词，d\$ 一下就删到行尾了。以前我要按着退格键一下一下地删，像在喂一只慢吞吞的兔子。”

“d\$ 是直接清空整行末尾到行尾的这句话，等你学到查找命令，d/ 还能删到某个词的位置。”

### 6.3 撤销与重做：后悔药

Vim 里的撤销和重做，比普通编辑器强太多。在普通编辑器（比如记事本或 VS Code）里，撤销是一维的：按 Ctrl+Z 一步步倒退，再按 Ctrl+Y 一步步前进。Vim 的撤销树是多维的，不过你目前不需要深入了解——你只需要记住三个基本命令：

| 命令       | 效果                          |
|----------|-----------------------------|
| u        | 撤销上一次修改                     |
| U        | 撤销对当前行的所有修改（一步回到这行被修改之前的状态） |
| Ctrl + r | 重做（撤销刚才的撤销）                 |

试试：删了一行（dd），按 u —— 它回来了。

再试：在这一行上做了好几个操作——删了一个词、改了一个字母、又粘贴了一段文字。按 `U`（大写）——整行一次性回到最初的状态。

再试：删了一行，按 `u` 撤销，按 `Ctrl + r` —— 又没了。

`u` 是 *undo*（撤销），`U` 是撤销整行的所有修改（小步退 vs 一步回到原点），`Ctrl+r` 是 *redo*（重做）。如果你不小心撤销多了，`Ctrl+r` 是反悔药。

Kate 说：“就这？跟记事本的 `Ctrl+Z` 一样啊。”

“表面一样。但 Vim 的 `u` 可以撤销很多次——不只是‘输入的文字’，还包括删除、替换、缩进……所有编辑操作。而且 `U`（大写）可以一次性撤销你在一整行上做的所有修改。删一行、改一个词、又粘贴了一段——按一下 `U`，全部回退。`u` 是小步退，`U` 是一步回到原点。另外，记事本撤销多了会丢失历史，Vim 的撤销历史更可靠。”

## 6.4 删除并进入插入模式：`c` 命令

`c` 是 Vim 里的 **change**（修改）命令。它做的事情是：**删除指定的内容，然后自动进入插入模式**，等着你输入新的内容。用 `c` 代替 `d`，相当于把“删除”和“切换到插入模式”合并成一步。

| 组合               | 效果                                    |
|------------------|---------------------------------------|
| <code>cw</code>  | 删除到下一个单词开头，进入插入模式                     |
| <code>cb</code>  | 删除到上一个单词开头，进入插入模式                     |
| <code>c\$</code> | 删除到行尾，进入插入模式                          |
| <code>c0</code>  | 删除到行首，进入插入模式                          |
| <code>cc</code>  | 删除整行，进入插入模式（相当于 <code>dd + i</code> ） |

`cw` 是新手最常用的修改命令。它让你**原地替换一个单词**：

```
Hello world
```

光标停在 `world` 的 `w` 上，按 `cw` —— `world` 消失了，你进入了插入模式，光标停在 `Hello` 后面。输入 `Vim`，按 `ESC`。

```
Hello world → Hello Vim。
```

**两步：`cw` + 输入新词。** 如果是 `dw + i`，需要三步（`dw`、`i`、输入新词）。`cw` 把删除和切换插入模式合并成一步，省了一次按键。

Kate 试了 `cw`，说：“我懂了，`cw` 就是‘干掉这个词，然后我来写个新的’。我以前都是删掉再按 `i`，现在是删掉的同时按 `i`。”

## 6.5 替换单个字符：r 命令

r 是 **replace**（替换）命令。它把光标下的字符替换成你按的下一个字符，然后自动回到普通模式。

试试：

```
Hello
```

光标停在第一个 l 上，按 r，然后按 x —— l 变成 x，变成 HeXlo。

**一步到位：**按 r + 新字符 = 替换。不需要进入插入模式、删掉旧字符、输入新字符、退出插入模式那四步操作。

Kate 说：“这个好。有时候我写代码打错一个字母，之前都是按 i 进去改，然后再按 ESC 出来。现在 r 直接一步搞定。”

## 6.6 本课小结

能删、能改、能反悔——这就是 Vim 最常用的编辑方式。以后你写代码、改配置、修 bug，这套动作会日复一日地出现在你的手指上。

| 命令       | 效果                |
|----------|-------------------|
| x        | 删除光标下的一个字符        |
| X        | 删除光标前的一个字符        |
| dd       | 删除整行              |
| D        | 删除到行尾             |
| dw       | 删除到下一个单词开头        |
| d\$      | 删除到行尾             |
| u        | 撤销上一次修改           |
| U        | 撤销对当前行的所有修改       |
| Ctrl + r | 重做                |
| cw       | 删除到下一个单词开头，进入插入模式 |
| cc       | 删除整行，进入插入模式       |
| r        | 替换光标下的字符          |

**一句话记住这节课：**d 是“干掉”，u 是“反悔”，U 是“整行反悔”，cw 是“重写”，r 是“纠正”——记住这五条，Vim 编辑最核心的操作你都会了。

## 6.7 本课作业

1. 打开 Vim，新建一个文件
  2. 输入几行文字，然后用 `dd`、`dw`、`d$` 各删除一些内容
  3. 用 `u` 撤销每一步，再按 `Ctrl+r` 重做，观察屏幕变化
  4. 在某一整行上做多个操作（删词、改字、粘贴），然后用 `U` 一次性回退整行
  5. 找一个词，用 `cw` 把它改成另一个词
  6. 找一个写错的字母，用 `r` 修正它
- 守夜者宣言：**过去三课你学了移动、插入、删除、撤销。你现在已经能——打开文件、在任意位置插入文字、删除任意内容、改错、反悔。这是一套完整的“编辑工作流”。你已经不是在“学 Vim”，你是在“用 Vim”了。

下一课：复制、粘贴与缩进。

## 第七课：复制、粘贴与缩进 —— 搬砖的艺术

**这一课你将学会：**

- ✓ 复制文本（`y` 家族）
- ✓ 粘贴文本（`p` 和 `P`）
- ✓ 调整缩进（`>>` 和 `<<`）

**完成后，你不再需要鼠标来“搬东西”。** 所有复制粘贴，都在主键盘区完成。

上一课你学会了删除、撤销、修改。这一课我们学“搬东西”——复制、粘贴、缩进。

在普通编辑器里，复制粘贴靠鼠标右键或 `Ctrl+C / Ctrl+V`。在 Vim 里，**复制和粘贴是两个独立的命令**。但先别急，这一课我们只学最常用的操作：**复制一行、粘贴一行、调整缩进**。够你应付 80% 的场景了。

### 7.1 复制（Yank）：不是 copy，是 yank

Vim 里的复制不叫 copy，叫 **yank**（猛拉）。命令是 `y`。想象你“猛拉”了一段文本，存到了剪贴板里。

| 命令               | 效果         | 口令                                        |
|------------------|------------|-------------------------------------------|
| <code>yy</code>  | 复制当前整行     | “复制这一行”（双 <code>y</code> ）                |
| <code>yw</code>  | 复制到下一个单词开头 | <code>y</code> + <code>w</code> = 复制到下一个词 |
| <code>y\$</code> | 复制到行尾      | <code>y</code> + <code>\$</code> = 复制到行尾  |
| <code>y0</code>  | 复制到行首      | <code>y</code> + <code>0</code> = 复制到行首   |
| <code>yG</code>  | 复制到文件末尾    | <code>y</code> + <code>G</code> = 复制到文件尾  |
| <code>ygg</code> | 复制到文件开头    | <code>y</code> + <code>gg</code> = 复制到文件头 |

跟 `d` 一样的逻辑：`y` + 移动命令 = 复制从当前位置到目标位置的内容。

试试：

```
Hello world from Vim
```

光标停在 `Hello` 的 `H` 上：

- 按 `yw` → 复制 `Hello`
- 按 `y$` → 复制整行（从光标到行尾）
- 按 `yy` → 复制整行（不管光标在行内哪个位置）

## 7.2 粘贴 (Put) ：放在哪？

复制完了，怎么贴出来？

| 命令                  | 效果              | 口令    |
|---------------------|-----------------|-------|
| <code>p</code> (小写) | 在光标 <b>后</b> 粘贴 | “往后贴” |
| <code>P</code> (大写) | 在光标 <b>前</b> 粘贴 | “往前贴” |

试试：

```
Line 1
Line 2
```

光标停在第 2 行的任意位置：

- 按 `yy` 复制第 2 行
- 按 `k` 移到第 1 行
- 按 `p` → 在第 1 行**后面**粘贴，变成：

```
Line 1
Line 2
Line 2
```

如果按 `P` (大写)，会在第 1 行**前面**粘贴：

```
Line 2
Line 1
Line 2
```

Kate 说：“`p` 和 `P` 的区别我记不住。”

“小写 `p` 是往后放，大写 `P` 是往前放。你想象光标是一个排队的人——小 `p` 是他转身站到你后面去，大 `P` 是他直接插队到你前面。”

“那我是小 P 还是大 P？”

“你用小写 p 就够了。等你遇到‘我想把这段文字贴在前面而不是后面’的时候，你会主动想起还有个大写 P。”

## 7.3 缩进：让代码整整齐齐

写代码的时候，缩进是家常便饭。Vim 里缩进和反缩进非常简单：

| 命令 | 效果        | 口令    |
|----|-----------|-------|
| >> | 当前行向右缩进一次 | “向右推” |
| << | 当前行向左缩进一次 | “向左拉” |

试试：

这行文字

光标停在这行，按 >> ——整行向右移动一个缩进单位（默认是 2 或 4 个空格）。按 << ——缩回来。

**多行缩进：**用数字重复：

- 3>> → 从当前行开始，连续 3 行向右缩进
- 5<< → 从当前行开始，连续 5 行向左缩进

Kate 试了一下 >>，说：“以前我是按 Tab 键缩进。Vim 里按 >> 才缩进，按 Tab 反而会跳到下一格？这得习惯。”

“Tab 在 Vi 诞生的 1976 年就已经有了自己的含义——在插入模式里输入一个制表符。Bill Joy 不能用 Tab 来做缩进，所以他专门用了 >>。习惯之后你会发现 >> 更好——因为它永远不会跟你按 i 时误按 Tab 混在一起。”

## 7.4 组合拳：复制、粘贴、缩进一块练

场景：你在写一个配置文件，有三行内容，格式不对，想调整。

第一行内容  
第二行内容  
第三行内容

任务：把“第三行内容”复制一份，放在第二行后面，然后把所有行缩进一次。

步骤：

1. 光标移到第三行 → yy 复制
2. k 移到第二行 → p 粘贴（第二行后面多了一行“第三行内容”）



3. `gg` 移到第一行 → `3>>` 让三行都缩进
4. `:wq` 保存退出

**三件事，六个命令。** 如果让你在鼠标编辑器里做：选中第三行 → `Ctrl+C` → 鼠标点到第二行后面 → `Ctrl+V` → 选中三行 → 按 `Tab` → 保存。哪个快？你自己品。

## 7.5 本节课你能拿走的

| 功能 | 命令                                                               |
|----|------------------------------------------------------------------|
| 复制 | <code>yy</code> (行)、 <code>yw</code> (词)、 <code>y\$</code> (到行尾) |
| 粘贴 | <code>p</code> (往后)、 <code>P</code> (往前)                         |
| 缩进 | <code>&gt;&gt;</code> (右)、 <code>&lt;&lt;</code> (左)             |

你先记住这三个组合：

- 复制一行 → `yy`
- 粘贴到下一行 → `p`
- 缩进当前行 → `>>`

其他的，用到再说。

## 7.6 本课作业

1. 打开 Vim，新建一个文件
2. 输入 5 行文字
3. 复制第 3 行，粘贴到第 1 行后面
4. 复制第 5 行，粘贴到第 2 行后面
5. 用 `>>` 把第 1 行缩进一次，然后用 `.` 命令把剩下 4 行也缩进（或者直接用 `5>>`）
6. 保存退出

**守夜者宣言：** 复制粘贴是编辑器里最基础的操作。Vim 让它变得更快——你的右手不需要离开主键盘区去摸鼠标。所有操作都在 `y`、`p`、`>` 这三个键上完成。这一课的练习，就是让这三个键变成你的肌肉记忆。

下一课：搜索与替换。

## 第八课：文件信息、跳转、定位括号和缩进 —— 别迷路

这一课你将学会：

- ✓ 看一眼就知道文件有多大、光标在哪（`Ctrl+g`）
- ✓ 在文件里快速“瞬移”（`gg`、`G`、`0`、`$`）
- ✓ 一秒钟找到配对的括号（`%`）

☑ 批量调整缩进（>>、<< 的进阶用法）

完成后，你在文件里移动不再是“走”，而是“飞”。

前七课你已经学会了在 Vim 里“走路”（hjk1）、“跑步”（w、b）和“开车”（.、u）。这一课我们学“瞬移”——让你在文件里像打游戏一样，想去哪就去哪。

## 8.1 看一眼就知道自己在哪：Ctrl+g

你在一个 500 行的文件里编辑了半个小时。这时候你最想知道的是三件事：这是第几行？这个文件总共有多少行？光标在文件的什么位置（百分比）？

按 **Ctrl + g**。屏幕底部会显示一行信息：

```
"笔记.md" 第 42 行，共 128 行 --23%--
```

中文意思是：“你正在编辑的笔记.md，光标在第 42 行，文件总共 128 行，你大概在文件 23% 的位置。”

Kate 按了 **Ctrl+g**，说：“原来 Vim 也有‘进度条’。”

“对。而且它不会弹出来挡住你。就在底部一行，看一眼就消失。”

## 8.2 定位匹配括号：%（写代码的救命键）

你写代码的时候，最怕的事情之一是：括号不匹配。

```
function test() {  
  if (condition) {  
    console.log("hello");  
  }  
}
```

你看到第 2 行有一个 {，想知道它的 } 在哪。用肉眼一行一行数？太慢了。

把光标停在 { 上，按 %。光标会跳到配对的 } 上。再按一次 %，它又跳回来。

% 能匹配的符号：

- () 圆括号
- {} 花括号
- [] 方括号
- /\* \*/ 注释块（Vim 会跳转到配对的注释结束符）

试试：

```
def hello():  
    print("world")
```

光标停在 ( 上，按 % → 跳到 )。再按 % → 跳回来。

Kate 说：“我写 Python 的时候经常少一个括号。用 % 能检查有没有配对？”

“对。光标停在左括号上，按 % ——如果光标没动，说明没有配对的右括号。你就知道少写了一个。”

### 8.3 缩进调整：你之前学的只是皮毛

第 7 课你学了 >> 和 << 缩进当前行。这一课我们拓展一下——**批量缩进整段代码**。

```
if True:  
print("这行没有缩进")  
print("这行也没有缩进")
```

你想把 print 两行都缩进一次。

**方法一：数字重复（不需要可视模式，你已经知道）**

2>> → 从当前行开始，连续 2 行向右缩进

**方法二：可视模式 + >（选中再缩进，更直观）**

按 v 进入行可视模式，用 j 选中两行，然后按 > 缩进一次。按 . 可以再缩进一次。

可视模式的具体用法我们会在选修课展开，这里先知道有这个操作就行。

### 8.5 本节课你能拿走的

| 功能      | 命令     |
|---------|--------|
| 看一眼自己在哪 | Ctrl+g |
| 跳到匹配的括号 | %      |
| 缩进当前行   | >>     |
| 反缩进当前行  | <<     |

### 8.6 本课作业

1. 打开一个你已经写过的 Vim 笔记文件（或者随便复制一段 20 行以上的文字）

2. 按 `Ctrl+g` 查看文件信息，写下：这个文件多少行？光标在百分之几的位置？
3. 按 `gg` 飞到文件开头，按 `G` 飞到文件末尾，来回 5 次，让手指记住这两个键
4. 找到一行有括号的代码，把光标停在 `(` 或 `{` 上，按 `%` 跳到配对括号，再按一次跳回来
5. 找到一段没有缩进的代码，用 `>>` 缩进当前行，用 `.` 重复缩进下一行

**守夜者宣言：** 第七课你学会了复制粘贴，第八课你学会了飞。你现在已经能在文件里“想去哪就去哪”——不是慢慢走，是瞬移。加上括号定位，你已经不会在代码里迷路了。

下一课：搜索、替换 —— 在文件中“抓人”

## 第九课：搜索、替换 —— 在文件中“抓人”

这一课你将学会：

- ✓ 用 `/` 关键词 在文件里“抓人”
- ✓ 按 `n` 和 `N` 在匹配项之间来回跳
- ✓ 替换当前行的第一个词（`:s/old/new/`）
- ✓ 替换当前行的所有词（`:s/old/new/g`）
- ✓ 替换全文（`:%s/old/new/g`）

完成后，你不再需要滚轮+肉眼找东西。想找什么，直接“抓”。

前八课你学会了移动、编辑、复制、缩进。你现在已经能在 Vim 里“写”和“改”了。这一课我们学“找”——在文件里搜索关键词，然后把它们批量换掉。

如果你之前用过 `Ctrl+F` 搜索，这一课就是你熟悉的“搜索 + 替换”的 Vim 版——但快得多，而且全程不碰鼠标。

### 9.1 搜索：告诉 Vim “帮我找这个”

在普通模式下，按 `/`，然后输入你想找的内容，按回车。

```
/守夜者
```

Vim 会高亮显示文件里所有“守夜者”这个词。光标会自动跳到第一个匹配的位置。

*Kate 在笔记里搜了一下“Vim”，所有“Vim”都被高亮标出来了。她说：“这跟 `Ctrl+F` 差不多嘛。”*

“表面一样。但区别是——你不用伸手去摸鼠标，不用在右上角找那个搜索框。你的手一直放在主键盘区，按 `/` 就能搜。”\*

### 9.2 翻找：在搜索结果里“跳来跳去”

搜完之后，光标停在第一个匹配项上。想去看下一个匹配项？

| 命令 | 效果                | 口令              |
|----|-------------------|-----------------|
| n  | 跳到 <b>下一个</b> 匹配项 | n = next        |
| N  | 跳到 <b>上一个</b> 匹配项 | Shift + n = 倒着走 |

试试：

- 搜 /vim → 光标跳到第一个 vim
- 按 n → 跳到第二个 vim
- 按 n → 跳到第三个 vim
- 按 N → 跳回第二个 vim

Kate 说：“n 是‘下一个’，N 是‘上一个’。那大写 N 就是‘南’——往南走就是回头，我记住了。”

“也行。只要能记住，用什么谐音都行。”\*

### 9.3 替换：把“旧”换成“新”

搜索是“找”。替换是“找到，然后换掉”。

在普通模式下，输入 :s/旧词/新词/ ，按回车。

```
:s/夜行者/守夜者/
```

效果：把当前行**第一个**出现的“守夜者”替换成“夜行者”。

如果一行里有多“守夜者”， / 只换第一个。剩下的不动。

### 9.4 批量替换：全部换掉（g 标志）

加一个 g（global），就会替换当前行**所有**匹配项：

```
:s/夜行者/守夜者/g
```

g 的意思是“**这一行**里所有的”。不加 g 只换第一个，加了全换。

### 9.5 全文替换：整个文件一起换（% 范围）

:s 默认只对当前行生效。如果你想对整个文件生效，在前面加一个 %：

```
:%s/夜行者/守夜者/g
```

% 代表“整个文件”。这一句的意思是：“在整个文件里，把所有的‘夜行者’换成‘守夜者’。”

这可能是你以后最常用的替换命令。

Kate 试了一下 `:%s/VIM/Vim/g`，整个文件里所有的“VIM”都变成了“Vim”。她说：“这个好可怕。我要是输错了，整个文件就全毁了。”

“所以替换之前，先搜一下 `/Vim` 确认匹配数量。数量对上了，再执行替换。如果换错了，按 `u` 撤销，所有改动都会回来。`:%s` 是一个原子操作，Vim 会把它当作一次修改来记录——按一次 `u`，全部回退。”\*

## 9.6 补充技巧：确认替换（`c` 标志）

如果你不确定要不要全换，可以在命令后面加一个 `c`（confirm）：

```
:%s/夜行者/守夜者/gc
```

Vim 会逐个询问你：“这个要换吗？”按 `y` 换，按 `n` 跳过，按 `a` 全部换，按 `q` 退出。

## 9.7 本节课你能拿走的

| 功能        | 命令                      |
|-----------|-------------------------|
| 搜索        | <code>/关键词</code>       |
| 跳到下一个     | <code>n</code>          |
| 跳到上一个     | <code>N</code>          |
| 替换当前行的第一个 | <code>:s/旧/新/</code>    |
| 替换当前行的所有  | <code>:s/旧/新/g</code>   |
| 替换全文      | <code>:%s/旧/新/g</code>  |
| 替换全文，逐个确认 | <code>:%s/旧/新/gc</code> |

## 9.8 本课作业

1. 打开一个你之前写的 Vim 笔记文件（至少 20 行）
2. 随便搜一个词（比如 `vim`），用 `n` 和 `N` 跳几次
3. 把当前行的第一个“的”替换成“之”
4. 把当前行的所有“的”替换成“之”
5. 把全文所有的“的”替换成“之”（换完之后观察变化）
6. 按 `u` 撤销，把全文恢复原样
7. 再执行一次 `:%s/的/之/gc`，按 `a` 全部替换，观察“逐个确认”的体验

**守夜者宣言：** 搜索 + 替换，是你从“手动改文件”进化到“批量改文件”的最后一步。以后你改配置文件、批量修 bug、统一术语——全都用这一课的命令。你不再是“一行一行改”了，你是“一个命令全改完”。

下一课（最后一课）：哲学总结。

## 第十课：Vim 的哲学 —— 你站在了能看见风景的地方

**这一课你将学会：**

- ✓ 理解 Vim 为什么这样设计（不是反人类，是另有逻辑）
- ✓ 看懂“操作符 + 动作”这个核心公式
- ✓ 知道自己已经走到了哪里，以及接下来能往哪走

**完成后，你不再是“会用 Vim 的人”。你是“理解 Vim 的人”。**

前九课你学了一堆命令。这一课我们不学新命令——我们停下来，回头看一眼你走过的路。

十课之前，你打开 Vim 连退出都不会。现在你能打开文件、写字、保存、移动光标、删除、复制、粘贴、缩进、搜索、替换、撤销、重做。你已经在 Vim 里写完了这十课的笔记，保存、退出、再打开——内容还在。

这一课告诉你：**你刚才走过的路，背后有一套完整的哲学。**

### 10.1 普通模式是“家”，插入模式是“出门散步”

你前九课练得最多的动作是什么？是 **按 ESC**。

这不是巧合。Vim 的设计者把“普通模式”设计成默认状态——你的手放在键盘上，你什么也不做，Vim 就在普通模式里等你。你想打字？按 **i** 出门散步。写完了？按 **ESC** 回家。

**普通模式是“指挥”的地方。插入模式是“写字”的地方。**

新手最容易犯的错误是：一直待在插入模式里，用方向键移动、用退格键删改。如果那样用，Vim 跟记事本没区别。你学完这十课之后，你已经有了一种直觉：写完字立刻按 **ESC**，然后用 **h j k l** 移动，用 **dd** 删除，用 **y** 复制，用 **p** 粘贴——这些操作都在普通模式下完成，比插入模式里快得多。

### 10.2 Vim 是一门语言，不是一堆快捷键

你在前九课学了很多命令，它们不是“一串快捷键”——它们是一套语法。

这是 Vim 设计者最聪明的地方。他定义了几种基本的“操作”，又定义了几种“动作”，然后把它们组合起来，形成一套完整的“编辑语言”。

- **d** = delete（删除）
- **y** = yank（复制）
- **c** = change（修改）

- `>` = shift right (右缩进)

你学会了“动词”。这些动词，就是 Vim 的语法基础。

- `w` = 到下一个词
- `$` = 到行尾
- `G` = 到文件末尾
- `gg` = 到文件开头

你学会了“名词”，也就是动作的目标。

**动词 + 名词 = 一个完整的句子。**

```
d + w = 删掉这个词
y + $ = 复制到行尾
c + G = 修改到文件末尾
> + G = 缩进到文件末尾
```

你不需要记忆“`yw` 是复制单词”、“`dw` 是删除单词”、“`cw` 是修改单词”——你只需要知道 `y`、`d`、`c` 是什么，`w` 是什么，然后组合它们。

Vim 的语言模型，是你从第 6 课开始真正掌握的东西。那时候你可能没意识到，但你已经在用“动词+名词”的组合了。这是 Vim 最优雅的设计，也是你从新手到高手的核心分水岭。你现在已经跨过它了。

**学到这里，你开始理解 Vim 了。不是“背会了”，是“看懂了”。**你能看懂 `d$` 的含义，即使你从没见过它；你能解释 `yG` 是做什么的，即使你从没用过。

## 10.3 . 命令：你学会了“说一次，让它重复”

你在第五课学了一个键——`.`。它很不起眼，但它是整个 Vim 哲学里最高频、最核心的命令。

**你说一遍，Vim 帮你重复。**你不需要数“我要做多少次”，你只需要做一次，然后按 `.` 按到满意为止。这是 Vim 对你的承诺：“你做一遍，我帮你重复。”

所有 Vim 高手的肌肉记忆，都建立在这一个键上。当你看到 10 行代码需要在行尾加分号，你想到的不是“我按 10 次 `A`”，而是“我做一次，再按 9 次 `.`”。心算消失了，数数消失了，只剩手指的节奏。

## 10.4 你站在哪里了？

回头看你走过的路：

| 课     | 你学会了     | 你现在能做          |
|-------|----------|----------------|
| 第 1 课 | 启动、退出    | 活着进去，活着出来      |
| 第 2 课 | 打开、编辑、保存 | 在 Vim 里写一份真实笔记 |



| 课      | 你学会了                                                                                                          | 你现在能做           |
|--------|---------------------------------------------------------------------------------------------------------------|-----------------|
| 第 3 课  | <code>h j k l</code> 、 <code>g g</code> 、 <code>G</code> 、 <code>w</code> 、 <code>b</code>                    | 不用方向键移动光标       |
| 第 4 课  | <code>i</code> 、 <code>A</code> 、 <code>o</code> 、 <code>ESC</code>                                           | 在正确的位置打字，打完立刻回家 |
| 第 5 课  | <code>.</code> 命令                                                                                             | 做一次，重复无数次       |
| 第 6 课  | <code>d d</code> 、 <code>d w</code> 、 <code>u</code> 、 <code>U</code> 、 <code>c w</code> 、 <code>r</code>     | 删、改、撤、一步纠正      |
| 第 7 课  | <code>y y</code> 、 <code>p</code> 、 <code>&gt;&gt;</code>                                                     | 复制、粘贴、缩进        |
| 第 8 课  | <code>Ctrl + g</code> 、 <code>g g</code> 、 <code>G</code> 、 <code>0</code> 、 <code>\$</code> 、 <code>%</code> | 瞬移、定位括号、批量缩进    |
| 第 9 课  | <code>/</code> 、 <code>n</code> 、 <code>N</code> 、 <code>: s</code> 、 <code>: % s</code>                      | 搜索、批量替换         |
| 第 10 课 | Vim 的底层哲学                                                                                                     | 看懂了，不再只是“会用”    |

十课前，你是一个“听说过 Vim 但不敢用”的人。十课后，你在终端里打开 Vim，不再慌张；你知道 `h j k l` 在哪，知道写完按 `ESC`，知道 `g g` 飞到开头、`G` 飞到末尾；你知道 `d` 是动词、`w` 是名词，你能看懂 `y$` 和 `c w` 的含义——即使你从没见过它们。

## 10.5 下一站：选修课

必修课结束了，但 Vim 的路还很长。

如果你想继续走，这里有几个方向可以选：

- **可视模式** (`v`、`V`、`Ctrl + v`)：选中文本，然后删除、复制、缩进——视觉化的操作方式
- **寄存器** (`"`)：Vim 的剪贴板系统，可以同时保存多份不同的内容
- **宏** (`q`)：录制并重复一系列操作，比 `.` 命令更强大
- **查找替换进阶** (正则表达式)：`: % s / ... / ... / g` 的真正威力
- **插件管理**：把 Vim 变成你专属的 IDE

这十条路，你不需要都走。等你在某个场景里遇到痛点——比如“我想可视化选中文段再操作”或者“我想保存多份复制内容”——你再回来看选修课。

## 10.6 最后一句

Vim 的学习曲线像悬崖。

你刚开始爬的时候，陡得让你怀疑自己。但你上来了。你坚持了十课，每一课都做了练习。你手里现在拿着的，不是一张“Vim 速成证书”，是你十天的练习记录，是你敲过的每一条命令，是你改过的每一个错，是你按下的无数次 `ESC`。

你现在站在一个能看见风景的地方——不是山顶，但你已经在走了。

**守夜者宣言：** 十课之前，你问“Vim 怎么退出”。十课之后，你问“接下来还能去哪里”。这就是进步。这不是终点，是你真正开始使用 Vim 的起点。

## 第二部分：通往专家之路（选修）

**目标：**让你从“会用”变成“精通”。  
**读完前十课后，根据你的痛点，挑着读，不用全读。**

十节课之前，你问“怎么退出 Vim”。十节课之后，你问“怎么让 Vim 飞起来”。这就是进步。

必修课让你“能用”。选修课让你“好用”。

你不必全读。根据你的痛点，挑一个方向往下走就行。

### 我应该从哪个选修开始？

| 如果你遇到这些情况                    | 去读                                              |
|------------------------------|-------------------------------------------------|
| 每次删引号里的内容都要按好几下键             | <b>选修 1：文本对象</b> —— 一句“删掉引号里的东西”，Vim 秒懂         |
| 复制粘贴总被覆盖，剪贴板永远只有最后一件事        | <b>选修 2：寄存器</b> —— Vim 有 26 个剪贴板，你只是不知道而已       |
| 同样的操作要做 50 次，手指都麻了           | <b>选修 3：宏</b> —— 录一遍，让 Vim 帮你跑完剩下的              |
| 不想背命令，想自己“推导”出 Vim 的语法       | <b>选修 4：Vim 是一门语言</b> —— 你会了动词，现在该学介词和宾语了       |
| 想让 Vim 默认显示行号、自动缩进、用起来更顺手    | <b>选修 5：配置 .vimrc</b> —— 写一份配置文件，让 Vim 变成你喜欢的样子 |
| 想让 Vim 变成 IDE（代码补全、语法检查、文件树） | <b>选修 6：插件生态</b> —— 装几个插件，Vim 就变成了你想要的 IDE      |

### 选修1：文本对象 —— 操作的单位升级了

**这一课你将学会：**

- ✔ 什么是文本对象（你操作的不再是“字符”，而是“有意义的块”）
- ✔ 用 `dip` 删掉整个段落，即使光标只在这个段落中间
- ✔ 用 `ci"` 替换引号里的内容，不用管引号本身
- ✔ 操作符 + 文本对象 = 更高维度的“动词 + 名词”

**完成后，你不再“一个一个字符”地改东西。** 你直接告诉 Vim：“把这个括号里的内容换掉”、“把这段引号里的内容改了”、“把这个段落删掉”—— Vim 会自己找到边界。

你在必修课里学会了 `dw` 和 `cw` ——删掉/修改一个词。但如果你想把光标停留在段落中间，然后删掉整个段落？或者替换一对引号里的全部内容？`dw` 和 `cw` 就不够用了。

这就是文本对象出场的时刻。它让你操作的单位从“字符”升级到“单词、句子、段落、括号、引号”。

## 1.1 什么是文本对象？

文本对象 = 一段有边界的文本块。它是一段可以被 Vim 识别和操作的“内容单元”。

| 文本对象                              | 含义                   | 边界                                                       | 例子                                                                     |
|-----------------------------------|----------------------|----------------------------------------------------------|------------------------------------------------------------------------|
| <code>iw</code>                   | inner word           | 当前单词                                                     | “hello_world”按 <code>ciw</code> → 只替换 <code>hello_world</code> 本身，不碰空格 |
| <code>i"</code>                   | inner double quote   | 双引号内的内容                                                  | <code>"hello"</code> → <code>hello</code>                              |
| <code>i'</code>                   | inner single quote   | 单引号内的内容                                                  | <code>'world'</code> → <code>world</code>                              |
| <code>i(</code> 或 <code>ib</code> | inner parentheses    | 圆括号内的内容                                                  | <code>(hello)</code> → <code>hello</code>                              |
| <code>i{</code> 或 <code>iB</code> | inner braces         | 花括号内的内容                                                  | <code>{world}</code> → <code>world</code>                              |
| <code>i[</code>                   | inner brackets       | 方括号内的内容                                                  | <code>[vim]</code> → <code>vim</code>                                  |
| <code>i&lt;</code>                | inner angle brackets | 尖括号内的内容                                                  | <code>&lt;html&gt;</code> → <code>html</code>                          |
| <code>it</code>                   | inner tag            | HTML/XML 标签内的内容                                          | <code>&lt;p&gt;hello&lt;/p&gt;</code> → <code>hello</code>             |
| <code>ip</code>                   | inner paragraph      | 当前段落<br>(按空行分隔)                                          | 一段连续的文字                                                                |
| <code>is</code>                   | inner sentence       | 当前句子 (按 <code>.</code> <code>!</code> <code>?</code> 分隔) | 一个句号结束的句子                                                              |

多一个 **a** 版本：`aw` (a word) 会带上单词后面的一个空格，`a"` 会包含引号本身。`i` 是“只拿里面的”，`a` 是“连边界一起拿”。你先学会 `i` 系列就够用了。

Kate 在笔记里试了一下 `ciw` ——光标停在 `hello_world` 的 `w` 上，按 `ciw`，单词消失了，她进入了插入模式，直接输入 `Vim`，按 `ESC` 结束。她说：“我之前想改一个单词，得把光标移到开头，或者按 `dw` 删掉再按 `i` 写。`ciw` 是删词+开写同步完成，完全不用管光标在哪——只要还在这个单词里就行。”

“对。`ciw` 的核心优势就是：**光标不需要放在单词开头**。你只要在这一个单词的任何一个字母上，它都能找到边界。省掉了你移动光标到词首的那一步。”

## 1.2 操作符 + 文本对象 = 更高维度的组合

你在必修课里学过：`d` + `w` = 删掉一个词。那是“动词 + 移动命令”。

现在扩展一步：**动词 + 文本对象**，比“动词 + 移动命令”更精准、更稳定——因为你不需要提前移动光标到边界，Vim 会自动找到它。

动词 + 文本对象 = 对“这个块”执行“这个操作”

| 组合               | 效果       | 解释                                            |
|------------------|----------|-----------------------------------------------|
| <code>dip</code> | 删除整个段落   | 光标在段落内任意位置，按 <code>dip</code> ，整个段落消失         |
| <code>ci"</code> | 修改引号内的内容 | 光标在引号内，按 <code>ci"</code> ，引号保留，里面的内容清空，等你写新的 |
| <code>yi(</code> | 复制括号内的内容 | 光标在括号内，按 <code>yi(</code> ，括号内容被复制，光标位置不变     |
| <code>vip</code> | 选中整个段落   | 光标在段落内，按 <code>vip</code> ，整个段落被高亮选中          |
| <code>da[</code> | 删除方括号及内容 | 光标在方括号内，按 <code>da[</code> ，方括号和内容一起被删除       |

你不需要记住所有组合。你只需要知道这个公式存在。当你以后想“对某一块文本做某件事”的时候，你会自然地问自己：“这一块对应的文本对象是什么？”然后套进去。

## 1.3 实战：批量修改 HTML 标签属性

这是文本对象最经典的实战场景之一。

假设你有这样的 HTML 结构：

```
<div class="old-class" id="main">
  <p>欢迎来到守夜者的世界</p>
</div>
```

你想把 `class="old-class"` 改成 `class="new-class"`。

### 不用文本对象的方式：

1. 把光标移到 `old-class` 的 `o` 上
2. 用 `dw` 按词删：`old` → `dw`、`-` → `x`、`class` → `dw` ——三到四步，而且每个词的边界不同
3. 或者手动移动光标到 `o`，按 `d$` 删到行尾，然后重新输入——删多了
4. 或者按 `i` 进入插入模式，用退格键一个字符一个字符地删——最慢

### 用文本对象的方式：

1. 把光标移到 `old-class` 的任何一个字母上（`o`、`l`、`d`……任意一个）
2. 按 `ci"` —— 光标自动跳到最近的引号内，选中并清空 `old-class`，进入插入模式
3. 输入 `new-class`
4. 按 `ESC`

三步。不需要数词、不需要移动到开头、不需要按 `d$` 再重新补引号。

Kate 试了一下，说：“我不用管光标在哪？只要光标在引号里面，`ci`”就能找到这对引号的边界？”

“对。你不需要预先移动光标到引号开头。Vim 自己往前找最近的一对引号，选中它们之间的内容，清空，等你输入。这叫‘不精确瞄准，Vim替你定位’。”

## 1.4 文本对象能做的事（练习版）

打开 Vim，贴入这段文字：

```
"Vim is a powerful text editor"
function greet(name) {
    console.log("Hello, " + name + "!");
}
第一段文字。这是第二句。第三句。
```

按 `ESC` 确保在普通模式，然后按顺序练习：

- 1. 把光标移到引号里的任意一个字母上（比如 Vim 的 `i`），按 `ci`”，输入 `Neovim is even better`，按 `ESC` → 引号内的内容被替换
- 2. 把光标移到 `name` 的任意一个字母上，按 `ci(`，输入 `username`，按 `ESC` → 括号内的内容被替换
- 3. 把光标移到 `Hello` 所在的那对引号的任意一个字符上，按 `yi`” → 复制了引号内的内容（不复制引号本身）
- 4. 把光标移到第一段文字的任意位置，按 `dip` → 整个第一段消失
- 5. 按 `u` 撤销
- 6. 把光标移到第一段文字的任意位置，按 `vip` → 整个第一段被高亮选中

## 1.5 你能拿走的（就记这两个）

| 功能      | 命令                |
|---------|-------------------|
| 改引号里的内容 | <code>ci</code> ” |
| 删整个段落   | <code>dip</code>  |

你记不住 `i(`、`i[`、`i{` 也没关系。记住 `ci`” 和 `dip`，你已经在“操作单位”上升级了一个台阶。其他文本对象的用法，等你遇到具体场景自然需要时再回来查。

## 1.6 本课作业

- 1. 打开 Vim，新建一个文件
- 2. 输入一段带引号的文字（比如 `"Hello, Vim!"`）
- 3. 把光标移到引号里的任意位置，按 `ci`”，把内容改成 `"Hello, World!"`
- 4. 再输入一个段落（至少三行连续的文字，中间没有空行）

- 5. 把光标移到段落中的任意位置，按 `dip`，看段落是否消失
- 6. 撤销（`u`），恢复原样
- 7. 再按 `vip`，看整个段落是否被高亮选中（然后按 `ESC` 取消高亮）
- 8. 如果上面都完成了，找一段带括号的代码（比如 `function test(param)`），把光标移到括号内的任意位置，按 `ci(`，把 `param` 改成 `newParam`

**守夜者宣言：**必修课你学会了“操作字符”。选修课你学会了“操作块”。你现在是在 Vim 里“写代码”，而不是“敲字符”。这两者之间的区别，决定了你是“会用 Vim”还是“理解 Vim”。

## 选修2：寄存器 —— Vim 的“多个剪贴板”

这一课你将学会：

- ✓ 什么是寄存器（你不再只有一个剪贴板）
- ✓ 用命名寄存器保存多段文本，互不干扰
- ✓ 理解默认寄存器和复制专用寄存器的区别
- ✓ 实战：同时操作多段文本，想贴哪段贴哪段

**完成后，你可以在 Vim 里同时“记住”多段文字——像同时打开了好几个剪贴板。** 这是从“能用 Vim”到“用得很顺手”的重要一步。

你已经在必修课里用 `y` 复制、用 `p` 粘贴了。你用的是 Vim 的**默认寄存器**——它像一个只有一个槽位的剪贴板。你每复制一次，上次复制的内容就被覆盖了。

但如果你在修改一个配置文件的时候，需要同时操作多段内容呢？比如：复制一段日志、粘贴到另一处，再复制一段报错信息、粘贴到同一处——你不希望第二段复制把第一段覆盖掉。这时候你需要的就是寄存器。

寄存器是 Vim 的**多个剪贴板**。你可以把不同的内容存进不同的“抽屉”里，需要的时候再拿出来。

### 2.1 什么是寄存器？

寄存器就是 Vim 用来“暂存文本”的地方。你可以把它想象成一个有多个格子的工具箱，每一个格子就是一个独立的内存空间。

| 寄存器类型   | 符号                                | 什么时候用它                                | 你会用到吗        |
|---------|-----------------------------------|---------------------------------------|--------------|
| 匿名寄存器   | <code>""</code>                   | 你啥也没干的时候自动用的，存最近一次操作                  | 你一直在用（默认）    |
| 复制专用寄存器 | <code>"0</code>                   | 每次用 <code>y</code> 复制内容，Vim 会自动存一份到这里 | 刚复制完想粘的时候最有用 |
| 命名寄存器   | <code>"a</code> 到 <code>"z</code> | 你主动挑选的专用“抽屉”（像 26 个命名空间）              | 你想同时存多段内容的时候 |
| 系统剪贴板   | <code>"+</code>                   | 需要和外部程序（浏览器、终端）交换内容时                  | 跨程序粘贴时用到     |

你暂时只需要记住 **命名寄存器** 和 **复制专用寄存器** 怎么用。其他两个了解一下就行。

## 2.2 命名寄存器：你的 26 个专属“抽屉”

命名寄存器是 **a** 到 **z** 这 26 个字母。每个字母都是一个独立的抽屉。

为什么是 26 个？因为 QWERTY 键盘上有 26 个字母键。Vim 直接拿它们当寄存器名，不用单独设计一套符号系统——这很 Vim。

**往里放内容：**

"a + y + 移动命令 = 把内容存进 a 抽屉

比如，你要把当前行存进 **a** 寄存器：

"ayy

翻译成人话：“把这一行存到 **a** 抽屉里。”

**把内容贴出来：**

"a + p = 从 a 抽屉里取出来粘贴

比如，把 **a** 寄存器里的内容粘贴到光标位置：

"ap

翻译成人话：“从 **a** 抽屉里拿出来贴在后面。”

Kate 说：“所以 "ayy 和 "ap 是一条完整的‘存取路径’。我往 a 里放，从 a 里取。b、c、d .....每个字母都是一个独立剪贴板。”

**完整流程：**

1. 第一段要复制的内容 → 光标放上去 → "ayy （存到 a 抽屉）
2. 第二段要复制的内容 → 光标放上去 → "byy （存到 b 抽屉）
3. 走到第一个要粘贴的位置 → "ap （贴 a 抽屉的内容）
4. 走到第二个要粘贴的位置 → "bp （贴 b 抽屉的内容）

两个抽屉的内容互不干扰。

## 2.3 实战场景：同时操作多段日志

你现在手头有这样一个文件：

```
ERROR: Connection timeout (192.168.1.1)
ERROR: Disk space low (sda1)
INFO: Service started successfully
ERROR: Permission denied (config.yml)
INFO: Cache cleared
```

你需要把三段 ERROR 日志的 IP、磁盘分区、配置文件名分别提取出来，粘贴到另一个文件的特定位置。

### 不用寄存器的方式：

1. 复制第一段 ERROR 的 IP → 粘贴到目标位置
2. 复制第二段 ERROR 的磁盘分区 → 粘贴到目标位置
3. 复制第三段 ERROR 的文件名 → 粘贴到目标位置
4. 每复制一次，之前的剪贴板就被覆盖了，没法保存多份

### 用寄存器的方式：

1. 把光标移到 192.168.1.1 上 → "ayiw → 复制这个 IP 到 a 抽屉
2. 把光标移到 sda1 上 → "byiw → 复制分区名到 b 抽屉
3. 把光标移到 config.yml 上 → "cyiw → 复制文件名到 c 抽屉
4. 走到目标文件 → "ap → 贴 IP
5. 回到目标位置下一行 → "bp → 贴分区名
6. 再下一行 → "cp → 贴文件名

三段内容互不覆盖，你想什么时候贴就什么时候贴。等你最后粘贴完，a、b、c 三个抽屉里的内容仍然保持原样，随时可以再次引用，或者在下一次复制时被覆盖更新。

---

## 2.4 复制专用寄存器 "0：刚复制的内容去哪了？

你每次用 y 复制内容，Vim 做了两件事：

1. 存入匿名寄存器 ""（你按 p 时默认粘贴的）
2. 存入复制专用寄存器 "0

"0 和 "" 的区别是："0 只保存复制（y）的内容，不保存删除（d）或修改（c）的内容。如果你删了一行，匿名寄存器被覆盖了，但 "0 还保留着上一次复制的内容。

场景：你复制了一段代码（y），然后删了几行（d），然后想再粘贴之前复制的那段代码——按 "0p 就可以了，而不是 p。

Kate 说：“所以 "0 是一个专门用来保护‘我复制的内容’的寄存器。即使我中间删了其他东西，它也不会丢。”

“对。"0 是复制专用通道，不跟删除/修改通道共用。你按 p 用的是匿名通道，可能被覆盖；但 "0p 用的是复制专用通道，永远不会被删除操作覆盖。”\*



## 2.5 你能拿走的（就记这三个）

| 功能                    | 命令   |
|-----------------------|------|
| 把当前行存进 a 抽屉           | "ayy |
| 把 a 抽屉的内容贴出来          | "ap  |
| 粘贴最近一次复制的内容（即使中间删过东西） | "0p  |

记住这三个命令，你已经能同时操作多段内容了。至于 "b、"c、"d .....它们跟 "a 的用法完全一样，换字母就行。

## 2.6 本作业

1. 新建一个 Vim 文件，输入三行文字，分别标记为 A、B、C
2. 把 A 行复制到 a 寄存器（"ayy）
3. 把 B 行复制到 b 寄存器（"byy）
4. 把 C 行复制到 c 寄存器（"cyy）
5. 在文件末尾新建三行空行
6. 依次粘贴 a、b、c 的内容（"ap、"bp、"cp）
7. 验证：三行内容是否按 A、B、C 的顺序排列？

**守夜者宣言：**寄存器的本质是“把 Vim 的内存借给你用”。你用命名寄存器存内容时，Vim 是在帮你管理多段文本的临时存储。这是从“能用”到“会用”的关键一步。当你需要在代码仓库里同时搬运多段内容时，你会回来翻这一课。

## 选修3：宏 —— 录制操作，一键重放

这一课你将学会：

- ✓ 用 q 录制你的操作序列
- ✓ 用 @ 一键重放整段操作
- ✓ 实战：在 100 行代码前面都加上 // 注释

**完成后，你会拥有一项超能力——把“一连串操作”录下来，然后让 Vim 帮你重放成百上千次。你不是在“写代码”，你是在“指挥 Vim 替你写代码”。**

你在必修课里学了 . 命令——它重复的是**最近的一次修改**。但如果你的操作不止一步呢？比如：“跳到行首 → 插入 # → 跳到下一行 → 缩进 → 删除行尾空格”——整整五步操作。 . 只能重复最后一步，你按一次 u 也只退回一步。

这时候你需要的是**宏（Macro）**——把一整串操作录下来，然后当成一个命令来播放。

## 3.1 录制宏：把操作“录”进磁带里

宏的录制过程，像老式录音机按“录音键”开始录，按“停止键”结束录。

```
q + 寄存器名 → 开始录制（红点出现）  
做你的操作  
q → 停止录制（红点消失）
```

录制宏之前，你必须先决定一个“抽屉”（寄存器）来存放它——就是你在选修2里已经学会的 `a` 到 `z` 命名寄存器。

```
qa → 开始录制，存到 a 寄存器  
（你做的一系列操作：I、#、ESC、j.....）  
q → 停止录制
```

录制完成后，`a` 寄存器里存的**不是文本**，而是一串“操作指令”——和你选修2里存文本内容不同，它是可执行的程序序列。

录制时屏幕底部会出现 `录制中` 或 `recording` 字样，提醒你宏正在记录。录完按 `q` 停止，那个字样就消失了。

Kate 第一次录宏的时候，看到屏幕底部多了 `录制中` 三个字，说：“它在录我？所以我现在敲的每一个键都被它记下来了？”

“对。你在做，它在记。你做的所有移动、插入、删除、切换模式……全都被录成了指令序列，存在那个寄存器里。”

---

## 3.2 播放宏：按一次，做一次

```
@a → 播放一次 a 寄存器里录制的宏
```

第一次按 `@a`，Vim 会执行你录制的那串操作。第二次按 `@a`，它再做一遍。

如果你录了“给一行加注释”的宏，播放一次就能给一行加注释。如果你要加 100 行呢？

```
100@a → 播放 100 次 a 宏
```

或者先 `@a` 播放一次，然后按 `100.` 重复执行 `.` 命令——因为 `.` 会重复 `@a` 这个操作。但 `100@a` 更直接，一步到位。

宏的威力在于：你录制时只做一次，播放时可以做无数次。你自己录的是一段“示范动作”，Vim 帮你把这套动作反复执行成百上千次。

---

### 3.3 实战：给 100 行代码加注释

假设你有这样一段代码（或者你正在维护一个真实的代码文件，有 100 行）：

```
const name = "Vim"
let version = 9.1
function greet() {
  console.log("Hello, " + name)
}
// 后面还有 95 行.....
```

你想把每一行前面都加上 `//`，让它变成注释。

**手动做一遍：**

1. 按 `I` → 跳到行首，进入插入模式
2. 输入 `//` → 加注释符号
3. 按 `ESC` → 回到普通模式
4. 按 `j` → 移到下一行

这是四步操作。现在把它录下来：

1. 确保光标在第一行
2. 按 `qa` → 开始录制，存到 `a` 寄存器
3. 按 `I` → 进入插入模式，光标跳到行首
4. 输入 `//` → 加注释符号
5. 按 `ESC` → 回到普通模式
6. 按 `j` → 移到下一行（这是为了播放时能自动推进）
7. 按 `q` → 停止录制

现在你录好了一个名为 `a` 的宏，它的内容是：给当前行加注释，然后移到下一行。

播放 99 次：

```
99@a
```

观察屏幕：每一行都在你眼前依次加上 `//`，光标不断向下推进，直到所有行都被注释完。

*Kate 说：“我录了‘加注释+移下行’四步，然后一个命令执行了 99 次，100 行代码全注释完了。这比我逐行手动改快了 100 倍。”*

“而且你发现了吗？你录制的最后一步是 `j` ——这使得宏天然地支持‘逐行推进’。只要目标操作是逐行处理的，宏都可以这样设计，中间不需要手动干预位置。”\*

### 3.4 宏的设计技巧（新手最容易踩的三个坑）

| 误区            | 正确的做法                                     | 为什么                            |
|---------------|-------------------------------------------|--------------------------------|
| 录制时忘了最后一步移动   | 始终在宏末尾加一个 <code>j</code> 或 <code>n</code> | 否则宏每播一次都会停在原地，你无法连续推进到下一行      |
| 录制时用方向键，播放时卡住 | 用 <code>h j k l</code> ，不要用方向键            | Vim<br>宏里嵌入方向键可能在播放时因终端差异而表现异常 |
| 忘了第一行已经处理完了   | 从第二行开始播放，或者先处理好第一行再用 <code>99@a</code>    | 否则第一行会被执行两次，多出额外效果             |

#### 最保险的宏结构：

```
(处理当前行的工作) + j (移到下一行)
```

这样每播放一次，宏就会做完当前行的事，然后自动把光标推到下一行。你只需：先把第一行做完，再 `100@a`，它会依次处理剩下的所有行。

### 3.5 你能拿走的（就记这两个命令）

| 功能            | 命令                 |
|---------------|--------------------|
| 开始录制，存到 a 寄存器 | <code>qa</code>    |
| 播放一次 a 宏      | <code>@a</code>    |
| 播放 100 次      | <code>100@a</code> |

记住 `qa` 和 `@a` 就够了。`b`、`c`、`d` 的用法完全一样，换字母就行。

当你发现自己在重复做同一件事超过 3 次时，停下来思考一下：这个操作能录成宏吗？Vim 的宏哲学是：“**手动操作一次就够了，剩下的交给录制。**”如果你发现自己做了 4 遍同样的操作，你已经在浪费力气了——该用宏了。

### 3.6 本课作业

- 1. 用 Vim 新建一个文件，输入 10 行文字（每行随便写点东西）
- 2. 录一个宏 `qa`：给当前行行尾加一个 `!`，然后移动到下一行
- 3. 播放一次 `@a`，观察光标和文字的变化
- 4. 再播放一次，观察逐行推进的效果
- 5. 撤销（`u`），恢复原始状态
- 6. 录一个新宏 `qb`：给当前行行首加 `#`，移动到下一行
- 7. 用 `9@b` 给剩下的 9 行全部加注释

8. 保存退出

**守夜者宣言：**宏是 Vim 里“自动化”的最简形态。它不需要编程基础，不需要插件——你把操作顺序录下来，Vim 就能替你重复成千上万次。当你发现自己正在做第 4 遍同样的操作时，停下来录个宏。这 30 秒的录制，会帮你省下几分钟甚至几小时的重复劳动。

选修4：Vim 是一门语言 —— 命令组合的逻辑

这一课你将学会：

- ✓ Vim 的核心语法结构：动词 + 名词
- ✓ 为什么你不需要背“一千个命令”——你只需要知道几个动词和几个名词，然后组合
- ✓ 从“背命令”到“想命令”的思维转变

完成后，你面对一个陌生的编辑任务时，不再问“Vim 有没有这个命令”——你会问“我要用哪个动词 + 哪个名词”。这是从“按命令”到“想命令”的最后一跃。

如果你一路学完了必修课和选修课，你可能已经注意到一件事：

Vim 的命令不是“随机命名的”。它们背后有一套统一的逻辑。

你没背过 `y$`，但你看到它的时候能猜出它是“复制到行尾”。你没背过 `dG`，但你知道它是“删到文件末尾”。你没背过 `ci"`，但你看到它的时候能看出它是“修改引号内的内容”。

这不是你记忆力好。是 Vim 的设计本身有规律。

4.1 Vim 的语法：动词 + 名词

任何一门语言都有语法。Vim 的语法只有一种结构：

动词 + 名词

**动词** = 你要做什么事（操作符）

| 动词 | 含义            | 中文翻译          |
|----|---------------|---------------|
| d  | delete        | 删除            |
| y  | yank          | 复制            |
| c  | change        | 修改（删除后进入插入模式） |
| >  | shift right   | 右缩进           |
| <  | shift left    | 左缩进           |
| v  | visual select | 选中（进入可视模式）    |

名词 = 你要操作什么对象（动作命令 或 文本对象）

| 名词 | 含义               | 中文翻译        |
|----|------------------|-------------|
| w  | word             | 一个词         |
| \$ | end of line      | 到行尾         |
| G  | end of file      | 到文件末尾       |
| i" | inside quotes    | 双引号内的内容     |
| ip | inside paragraph | 当前段落        |
| t  | tag              | HTML 标签内的内容 |

动词 + 名词 = 完整的指令

| 组合           | 效果       | 翻译成人话       |
|--------------|----------|-------------|
| d + w = dw   | 删除一个词    | “删掉这个词”     |
| y + \$ = y\$ | 复制到行尾    | “从这复制到行尾”   |
| c + i" = ci" | 修改引号内的内容 | “把引号里的东西换掉” |
| > + G = >G   | 缩进到文件末尾  | “从这缩进到最后一行” |

Kate 盯着这张表看了一会儿，说：“所以 `d` 和 `w` 是独立的‘词’，它们可以组合成 `dw`。就像英文里的动词和名词可以组成句子？”

“对。Vim 的设计者把编辑操作拆成了两个维度：做什么 + 对什么做。你学会 5 个动词 + 10 个名词，就能组合出 50 个命令——而你只需要记住这 15 个词。”

这就是 Vim 的核心设计智慧：它不是教你去记“一千个快捷键”，而是教你一种“造句”的能力。

## 4.2 为什么 Vim 不需要背太多命令？

因为 Vim 不是“命令集”，它是“语法体系”。

| 做法  | 你需要记多少         | 遇到新任务怎么办  |
|-----|----------------|-----------|
| 背命令 | 几百个键位组合        | 查资料，死记硬背  |
| 学语法 | 5 个动词 + 20 个名词 | 自己组合，即兴造句 |

“学语法”意味着：你不需要记住 `d$` 和 `c$` 的区别——你只需要知道 `d` 和 `c` 的区别，以及 `$` 是什么。剩下的自动推导。

Vim 的操作符 + 动作命令模型，让你学会了“造句”的能力：遇到没见过的命令，你只需要记住它的**动词**和**名词**，就能推测出它的行为。

**举例：**你第一次看到 `yg_`，你不用背它——你知道 `y` 是复制，`g_` 是“到最后一个非空字符”，所以 `yg_` 是“复制到最后一个非空字符”。你从来没背过这个命令，但你第一次看到它就能大概猜出它做什么。

这叫“可推测性”。Vim 的所有命令都遵循同一套规则，所以你不必“背完”它。

### 4.3 从“按命令”到“想命令”

这是两种完全不同的心智模式：

| 阶段  | 你的思考过程                                                               | 你依赖的东西    |
|-----|----------------------------------------------------------------------|-----------|
| 按命令 | “我要删一个词 → 快捷键是什么来着？ <code>dw</code> ？”                               | 肌肉记忆 + 查表 |
| 想命令 | “我要删一个词 → 动词是 <code>d</code> ，名词是 <code>w</code> → <code>dw</code> ” | 语法推理      |

- “按命令”是新手阶段：你想起一个命令，然后执行它。如果忘了，就卡住了。
- “想命令”是高阶阶段：你想到一个意图，然后用语法结构自己“造”出对应的命令。即使你从没见过那个组合，你也能推理出来。

Vim 高手的脑子里没有“命令表”。他们的脑子里有“动词表”和“名词表”，以及一个组合规则。他们看到任何文本编辑任务，都能用这套规则“造句”。

### 4.4 实战：从“我想做什么”到“Vim 命令是什么”

你现在遇到一个任务：把文件里从当前位置到文件末尾的所有内容都改成大写。

你没背过这个命令。但你想一想：

- 动词是什么？**“改成大写” → 这个 Vim 里有一个动词叫 `gU`（变成大写），或者 `gu`（变成小写）
- 名词是什么？**“到文件末尾” → 名词是 `G`
- 组合：`gU` + `G` = `gUG`

你用 `gUG` 执行了它，成功了。你从来没背过 `gUG`，但你用 Vim 的语法推理出来了。

另一个例子：

- 任务：把当前单词改成大写
- 推理：动词是 `gU`，名词是 `iw`（inner word）
- 命令：`gUiw`

你不需要背 `gUiw`。你只需要知道 `gU` 是“变大写”，`iw` 是“当前单词”。然后组合。

### 4.5 Vim 的“动词表”扩展（你以后会遇到的）

你在必修课里学的是最常用的 3 个动词（`d`、`y`、`c`）。Vim 还有更多动词：

| 动词              | 含义    | 示例                              |
|-----------------|-------|---------------------------------|
| <code>gU</code> | 转大写   | <code>gUw</code> = 当前词变大写       |
| <code>gu</code> | 转小写   | <code>guw</code> = 当前词变小写       |
| <code>~</code>  | 反转大小写 | <code>~w</code> = 当前词大小写互换      |
| <code>=</code>  | 自动缩进  | <code>=G</code> = 从当前位置缩进到文件末尾  |
| <code>zf</code> | 折叠    | <code>zfG</code> = 从当前位置折叠到文件末尾 |
| <code>J</code>  | 合并行   | 把当前行和下一行合并                      |

你不需要背这个表。你只需要记住：Vim 里做**任何操作**，都遵循“动词 + 名词”的规则。你遇到一个新动词时，你自动知道怎么用它。

## 4.6 你走到了哪里

| 阶段       | 你的状态                                                                               |
|----------|------------------------------------------------------------------------------------|
| 第 1-2 课  | 背命令阶段： <code>i</code> 是插入、 <code>:w</code> 是保存                                     |
| 第 3-8 课  | 练习阶段：手开始记住 <code>hjk1</code> 、 <code>dd</code> 、 <code>yy</code> 、 <code>gg</code> |
| 第 9-10 课 | 理解阶段：你知道 <code>d</code> 和 <code>w</code> 是独立单元                                     |
| 你现在      | <b>语法阶段：你开始用动词 + 名词自己“造”命令</b>                                                     |

你刚进入这个阶段。你现在看到 `d$`，不再想“这是删除到行尾的快捷键”，而是想“`d` 是动词，`$` 是名词，组合起来就是删除到行尾”。这个思维转变，是 Vim 学习路上真正的里程碑。

从此刻起，你不再需要“背”新命令了。你只需要“学”新动词和“学”新名词，组合规则你已经会了。

## 4.7 本课作业

- 打开 Vim 终端
- 在不查找任何资料的情况下，猜出以下命令的效果（写完猜测，再实际验证）：
  - `d2w`
  - `y3G`
  - `gU$`
  - `cgg`
- 验证：输入一段文字，分别执行上述命令，观察是否符合你的猜测
- 记录：你猜对了几个？哪个命令和你的猜测最接近？

**守夜者宣言：** 你不再需要记住“一千个快捷键”。你只需要记住一套语法规则——动词 + 名词。其他的一切，都是你对这套规则的发挥。



Vim 学习之路上的最后一跃，不是“我记住了更多命令”，而是“我不需要记住命令了”。你现在站在这里。你以后看到任何一个 Vim 命令，都会问自己两个问题：“它的动词是什么？”“它的名词是什么？”

然后你会发现——你已经能看懂几乎所有 Vim 命令了。

## 选修5：配置 `.vimrc` —— 让 Vim 变成你的形状

这一课你将学会：

- ✓ `.vimrc` 是什么、放哪
- ✓ 理解每一行配置为什么存在——不只是复制粘贴
- ✓ 两种可选方案：Sublime 极简风 vs VSCode 现代风

完成后，你打开 Vim 不再是那个“灰扑扑的终端编辑器”了——它有了行号、语法高亮、舒服的缩进，甚至是你熟悉的配色。这是 Vim 真正属于你的开始。

你之前打开 Vim 时看到的界面，是它的“出厂状态”——没有行号、没有语法高亮、缩进是 8 个空格、灰底白字。就像一台刚拆封的电脑，没有装任何你喜欢用的软件。

但 Vim 允许你通过一个配置文件来定制它。这个文件叫 `.vimrc`（Vim 的“个人档案”）。你每次启动 Vim，它都会读取这个文件，按照你写的规则来调整自己。

### 5.1 `.vimrc` 是什么？放哪？

是什么？

`.vimrc` 是 Vim 的**启动配置文件**。它用 Vim 自己的命令语言写成。你打开 Vim 时，它会自动执行里面的每一行命令——就像你手动按顺序输入一样。

放哪？

| 操作系统          | 文件路径                               | 怎么找到它                                                                     |
|---------------|------------------------------------|---------------------------------------------------------------------------|
| Linux / Mac   | <code>~/.vimrc</code>              | 终端里输入 <code>cd ~</code> ，再输入 <code>ls -a</code> ，看有没有 <code>.vimrc</code> |
| Windows (WSL) | <code>~/.vimrc</code>              | 跟 Linux 一样（WSL 里的 Linux 环境）                                               |
| Windows (原生)  | <code>C:\Users\你的用户名\_vimrc</code> | 文件名叫 <code>_vimrc</code> ，不是 <code>.vimrc</code> （系统限制）                   |

如果这个文件不存在，你就自己创建一个。

Kate 问：“我打开 Vim 那么多次，从来没想过它的长相是可以改的。我以为编辑器长什么样就是什么样。”

“Vim 的设计哲学里有一条：编辑器应该适应你的习惯，而不是你适应它的出厂设置。`.vimrc` 就是 Vim 给你的一支笔，让你自己画出它的样子。”

## 5.2 逐行拆解：你需要的每一项配置，以及为什么需要它

下面是你在配置 `.vimrc` 时最常用到的设置。每一条我都会解释它**为什么存在**——你之前没有它的时候是什么体验，加上它之后能解决什么问题。

### `set number` —— 显示行号

没有它的时候，你不知道自己在第几行。Vim 报错说“第 47 行有语法错误”，你只能手动一行一行数。或者你用 `Ctrl+G` 查看当前行号，但那需要多按一次快捷键。加上 `set number`，每一行左边都有一个编号——你一眼就能找到第 47 行在哪。

### `syntax on` —— 开启语法高亮

这是最重要的一行配置。没有它，你打开的任何代码文件——Python、JavaScript、Markdown——全部是同一个颜色：灰白。关键词、字符串、注释、函数名——全都长得一模一样。开了语法高亮之后，不同语法元素自动显示不同颜色：关键字是橘色的，字符串是绿色的，注释是灰色的。你读代码不再需要逐字辨认——颜色已经帮你分好类了。

### `set tabstop=4` —— Tab 显示宽度

Vim 的出厂设置把 Tab 显示为 8 个空格的宽度。这意味着你每按一次 Tab，光标会跳出去很远——一段代码嵌套三层，半个屏幕被空白占满。把它改成 4，缩进更紧凑，代码在屏幕上能显示更多实际内容。

### `set shiftwidth=4` —— 缩进宽度

你在第七课学过 `>>` 用来缩进整行。`shiftwidth` 决定了 `>>` 每次缩进多少格。Vim 默认是 8 格——缩进一次太猛，代码像坐了火箭。改成 4 格之后，缩进更细腻。

### `set expandtab` —— Tab 转空格

这个设置让你在插入模式里按 Tab 键时，实际输入的是空格而不是制表符。为什么需要？因为混用 Tab 和空格是程序员之间永恒的战斗——你的代码在 VS Code 里缩进正常，到了 GitHub 上缩进全乱，因为不同编辑器对 Tab 的显示宽度不一样。用空格代替 Tab，你的缩进在任何地方看起来都一样。

### `set autoindent` —— 自动缩进

你在写代码时按下回车，光标会自动跳转到和上一行相同缩进的位置——不需要手动敲空格对齐。写嵌套代码时尤其有用：你在 `for` 循环里写 `if` 判断，在 `if` 判断里写函数调用——每一层的缩进都是自动的，回车就好。

### `set cursorline` —— 高亮当前行

你在一个几百行的文件里移动光标——`j`、`k`、`gg`、`G`——有时候会迷失自己在哪一行。开了 `cursorline` 之后，光标所在的那一行会有一个淡淡的背景色——就像笔记本上的荧光笔标记。

### `set hlsearch` —— 高亮搜索匹配项

你在第九课学过 `/` 搜索。每次搜索后，所有匹配的项都会高亮显示——你一眼就能看到文件里哪些位置出现了你搜的关键词。搜完之后想关掉高亮？输入 `:nohlsearch`（或简写 `:noh`）。在配置里加上这一行，你每次用 `/` 搜索都会自动高亮所有匹配项。

### `set incsearch` —— 边输入边搜索

和 `hlsearch` 搭配使用。你在搜索框里输入关键词时，Vim 会实时跳转到第一个匹配的位置——不需要等你按回车。你每输入一个字符，它就往前找一个匹配。这让你更快找到目标——通常输到第三个字母就已经定位到了。

### `set ignorecase` + `set smartcase` —— 智能大小写

这两个设置要放在一起理解。`ignorecase` 让你搜索时忽略大小写——搜 `hello` 能同时找到 `hello`、`Hello`、`HELLO`。但如果你在搜索时故意输入了大写字母——比如搜 `Hello` ——`smartcase` 会告诉 Vim：“这次搜索区分大小写，只找 `Hello`，不找 `hello`。”两条规则配合使用：默认忽略大小写，但当你输入大写时自动切换为精确匹配。

### `set clipboard=unnamed` —— 系统剪贴板

Vim 有自己的内部剪贴板（叫寄存器），和你的系统剪贴板是分离的。你在浏览器里复制了一段文字，按 `p` 是贴不进来的——因为 Vim 的寄存器里没有那段文字。反过来也一样——你在 Vim 里用 `yy` 复制了一行，转到浏览器里 `Ctrl+V`，贴出来的是别的东西。加上 `set clipboard=unnamed`，Vim 的默认寄存器会和系统剪贴板同步——你在 Vim 里复制的东西，可以直接贴到浏览器；你在网页上复制的东西，也可以直接在 Vim 里按 `p` 粘贴。

### `set mouse=a` —— 允许鼠标操作

这是给刚从 VS Code 转过来的你的过渡配置。有了它，你可以用鼠标点击来移动光标、用鼠标滚轮来滚动屏幕、用鼠标拖拽来调整窗口大小。虽然 Vim 的设计哲学是“手不离键盘”，但作为新手，偶尔摸一下鼠标不是罪过。等你以后肌肉记忆稳定了，可以再把这个配置关掉。

`inoremap` 是 Vim 的映射命令——把“按某个键”这个动作，替换成“执行一系列操作”。`inoremap` 的意思是“仅在插入模式下生效”。

以 `inoremap ( ()<Left>` 为例：当你在插入模式里输入 `(` 时，Vim 实际执行的是三个动作——先输入 `(`，然后光标左移一格，停在两个括号中间。你接着输入的内容会出现在括号里面。输入完按回车后，你需要手动调整缩进——Vim 的自动缩进会帮你把光标放在合适的位置。

Kate 听完这一串，说：“所以 `.vimrc` 不是什么神秘的黑魔法——它就是把我平时用 Vim 时每次手动输入的命令，全部写进一个文件里，让它自动执行。不需要背——用的时候查一下就行了。”

## 5.3 两种预设配置方案

下面给你准备了两种不同风格的配置方案。你不需要全看——挑一种你喜欢的风格，完整复制到 `.vimrc` 里，保存，然后输入 `:source ~/.vimrc` 让它生效。两种方案都用 **Monaco** 字体（macOS 和 Linux 自带，Windows 需要手动安装）。

### 方案一：Sublime 风格 —— 极简、冷静、灰底白字

适合喜欢干净、简洁、最小干扰的人。像 Sublime Text 的默认界面——灰底白字、极简配色、行号和高亮都有，但不会抢你的注意力。

```

" =====
" 守夜者 Vim 配置: Sublime 风格
" 特点: 灰底、白字、极简、无干扰
" 适合: 喜欢干净简洁的写作者
" =====

" ----- 视觉 -----
set number           " 行号
syntax on           " 语法高亮
set cursorline       " 高亮当前行
set showmatch        " 高亮匹配括号
set background=dark  " 深色背景模式
colorscheme desert   " 沙漠配色（灰底白字）

" ----- 字体 -----
set guifont=Monaco:h12 " GUI 字体（仅 MacVim/gVim 生效）
set linespace=4        " 行间距

" ----- 缩进 -----
set tabstop=4
set shiftwidth=4
set expandtab          " Tab 转空格
set autoindent

" ----- 搜索 -----
set hlsearch          " 高亮搜索结果
set incsearch         " 边输入边搜索
set ignorecase        " 搜索忽略大小写
set smartcase         " 有大写时区分大小写

" ----- 剪贴板 -----
set clipboard=unnamed " 系统剪贴板

" ----- 鼠标 -----
set mouse=a           " 允许鼠标

" ----- 状态 -----
set laststatus=2      " 始终显示状态栏
set ruler             " 状态栏显示光标位置

" ----- 括号自动补全 -----
inoremap ( (<Left>
inoremap [ [<Left>
inoremap { {<Left>
inoremap " "<Left>
inoremap ' '<Left>
inoremap ` ``<Left>

```

## 方案二：VSCode 风格 —— 现代、彩色、暗底亮字

适合从 VSCode 转过来的你。配色接近 VSCode 默认的 Dark+ 主题——深色背景、关键字和字符串用亮色高亮、状态栏更详细。

```

" =====
" 守夜者 Vim 配置: VSCode 风格
" 特点: 暗底、彩字、现代感
" 适合: 从 VSCode 过来的写作者
" =====

" ----- 视觉 -----
set number           " 行号
syntax on           " 语法高亮
set cursorline       " 高亮当前行
set showmatch        " 高亮匹配括号
set background=dark  " 深色背景模式
colorscheme slate     " 暗色主题 (接近 VS Code Dark+)

" ----- 字体 -----
set guifont=Monaco:h12 " GUI 字体 (仅 MacVim/gVim 生效)
set linespace=4        " 行间距

" ----- 缩进 -----
set tabstop=4
set shiftwidth=4
set expandtab
set autoindent
set smartindent       " 智能缩进 (自动识别代码结构)

" ----- 搜索 -----
set hlsearch
set incsearch
set ignorecase
set smartcase

" ----- 剪贴板 -----
set clipboard=unnamed

" ----- 鼠标 -----
set mouse=a

" ----- 状态 -----
set laststatus=2
set ruler
set showcmd          " 底部显示当前输入的命令
set statusline=%F%m%r%h%w\ [%{&ff}]\ %l:%c\ %p%%

" ----- 括号自动补全 -----
inoremap ( (<Left>
inoremap [ [<Left>
inoremap { {<Left>
inoremap " "<Left>
inoremap ' '<Left>
inoremap ` `<Left>

```

两种方案的区别主要在 `colorscheme` 和 `statusline`。Sublime 风格用 `desert`（灰底白字，极简），VSCode 风格用 `slate`（暗底彩字，状态栏更丰富）。其他基础配置完全一样——行号、缩进、搜索、剪贴板、鼠标。

Kate 试了 VSCode 风格，看着新的界面说：“这好像我熟悉的 VS Code——深色背景、彩色的代码、行号在左边。但我还是在用 Vim 的命令编辑。外观接近 VS Code，内核还是 Vim。”

“这就是 `.vimrc` 给你的一支笔——你把 Vim 涂成了你习惯的颜色。”

## 5.4 替换配色主题

你不一定喜欢 `desert` 或 `slate`。Vim 自带了很多主题，你可以自己浏览：

- 1. 打开 Vim，输入 `:colorscheme`（后面加一个空格）
- 2. 按 `Tab` 键，Vim 会循环显示所有可用主题名
- 3. 选一个你喜欢的名字，替换 `.vimrc` 里的 `colorscheme` 那一行
- 4. 输入 `:source ~/.vimrc` 让配置立刻生效

常用主题速查：

| 主题名                  | 风格      | 适合         |
|----------------------|---------|------------|
| <code>desert</code>  | 灰底白字，极简 | Sublime 风格 |
| <code>slate</code>   | 暗底彩字，现代 | VSCode 风格  |
| <code>elflord</code> | 深紫底白字   | 喜欢暗色但不想太黑  |
| <code>ron</code>     | 深蓝底白字   | 看久了不累      |
| <code>koehler</code> | 黑底彩字    | 经典 Vim 风格  |
| <code>morning</code> | 白底黑字    | 白天光线强的时候   |

Kate 按了 `Tab`，看着主题名一个一个切换，说：“原来 Vim 自带这么多皮肤。我一个一个换着试——就像在给编辑器穿衣服。”

## 5.5 配置生效有两种方式

你编辑完 `.vimrc` 后，有两种方式让它生效：

- 1. 关闭 Vim，重新打开（全部重新加载）
- 2. 在 Vim 里输入 `:source ~/.vimrc`（立即加载，无需重启）

建议你采用第二种方式，一边编辑一边加载，可以实时看到变化，直到找到自己最舒服的配置组合。

## 5.6 本课作业

1. 找到你的 `.vimrc` 文件（没有就新建一个）
2. 从上面两种方案里选一个你喜欢的，完整复制进去，保存
3. 打开 Vim，输入 `:source ~/.vimrc`，观察界面的变化
4. 用 `:colorscheme` + Tab 浏览其他主题，换到你最喜欢的配色
5. 打开一个 Markdown 文件，看看语法高亮是否生效，行号是否显示

**守夜者宣言：** Vim 的默认配置，是 1976 年的产物。它不是为了让你“舒服”而设计的，是为了在极差的终端上也能跑起来。而 `.vimrc` 是给你的笔——让你的 Vim 从一台别人造好的工具，变成一副属于你自己的兵器。你打开 Vim 的时候，不再是面对一个陌生人，而是回到一个你亲手布置过的工作台。

## 选修6：插件生态 —— 把 Vim 变成 IDE

这一课你将学会：

- ✅ 什么是插件管理器，为什么你需要它（一个命令装/卸/更新所有插件）
- ✅ 用 vim-plug 装插件：三行代码搞定
- ✅ 三款必装插件：NERDTree（文件树）、coc.nvim（智能补全）、fzf（模糊搜索）
- ✅ 守夜者忠告：插件是工具，不是收藏品

**完成后，你的 Vim 不再只是一个终端编辑器——它能显示文件树、自动补全代码、秒级搜索任何文件。** 你可以在 Vim 里完成一个完整项目的编写，而不用切换到其他编辑器。

你现在已经有一个配置好的 `.vimrc` 了：有行号、有语法高亮、有舒服的缩进。但 Vim 的“出厂形态”缺几样东西：文件树、代码补全、快速跳转文件。

这几样功能，需要装插件来实现。但 Vim 本身没有“一键安装插件”的功能——你需要一个**插件管理器**来帮你完成这件事。

### 6.1 插件管理器：你的一站式插件商店

插件管理器就是一个帮你做三件事的工具：

1. 从 GitHub 等仓库下载插件代码，放到正确的位置
2. 在 `.vimrc` 里声明“我要用这个插件”
3. 一键更新所有插件、一键卸载不需要的插件

你只需要在 `.vimrc` 里写一行插件名，插件管理器就会帮你搞定剩下的所有事。

Kate 问：“那我是不是得像装软件一样，一个一个去下载？”

“不用。你在 `.vimrc` 里写一行 `Plug 'preservim/nerdtree'`，保存，然后输入 `:PlugInstall`。它会自动从 GitHub 下载、安装、配置好。你什么都不用做，除了输入那行命令。”



## 6.2 安装 vim-plug（三行命令）

### 方法一：终端命令（最快）

打开终端，粘贴下面这行命令，按回车：

```
curl -fLo ~/.vim/autoload/plug.vim --create-dirs \
https://raw.githubusercontent.com/junegunn/vim-plug/master/plug.vim
```

这行命令会从 GitHub 下载 `plug.vim` 文件，放到 Vim 的 `autoload` 目录。Vim 启动时会自动加载这个文件，让你可以在配置里使用 `Plug` 命令。

### 方法二：手动下载

1. 打开浏览器，访问 vim-plug 的 GitHub 仓库
2. 找到 `plug.vim` 文件，下载它
3. 把文件放到 `~/.vim/autoload/` 目录下（如果没有这个目录，自己创建）

安装完成后，在 `.vimrc` 中添加下面这三行结构：

```
" 插件开始标记
call plug#begin('~/.vim/plugged')

" 在这里列出你要装的插件
" Plug '插件名'

" 插件结束标记
call plug#end()
```

所有插件都写在 `plug#begin()` 和 `plug#end()` 之间。保存后，你每次添加新插件，运行 `:PlugInstall` 即可完成安装。

---

## 6.3 守夜者推荐插件清单（三款就够用）

你不需要装几十个插件。三款插件就能让你的 Vim 变成 IDE 级别的工具，而不会让启动变慢。

### 插件 1：NERDTree —— 文件树

Vim 默认只能看到当前文件的内容，没有“目录树”的概念。如果你在一个有几十个文件的项目里工作，你只能手动 `cd` 到目录、`ls` 列出文件、再用 `vim` 一个一个打开——每一步都在打断你的思考节奏。

NERDTree 在侧边栏显示整个项目的文件树。你按 `Ctrl+e` 就能切换文件树，按 `Enter` 打开文件，按 `r` 刷新目录。

Kate 装完 NERDTree 说：“我之前在 Vim 里打开文件，都是先开终端去 `ls` 看看有哪些文件，再复制文件名回来。现在直接边栏就能看到所有文件，点一下就开了。”

```
" ----- 插件管理器 -----
call plug#begin('~/.vim/plugged')

" 1. 文件树
Plug 'preservim/nerdtree'

call plug#end()

" ----- NERDTree 快捷键 -----
nnoremap <C-e> :NERDTreeToggle<CR> " Ctrl+e 切换文件树
```

## 插件 2: fzf —— 模糊搜索

你有一个项目文件夹，里面有几十上百个文件。想跳到一个叫 `config.yaml` 的文件里——你是去 NERDTree 里一层一层展开、肉眼找，还是直接输入文件名让 Vim 帮你找到它？

fzf 就是做这个的：输入几个字母，它实时过滤所有匹配的文件名。你按 `Ctrl+p`，输入 `conf`，所有包含 `conf` 的文件都列出来了，你选中它，按回车，文件就打开了。整个过程不到三秒。

```
" 2. 模糊搜索
Plug 'junegunn/fzf', { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'

" ----- fzf 快捷键 -----
nnoremap <C-p> :Files<CR> " Ctrl+p 搜索文件名
nnoremap <C-g> :Rg<CR> " Ctrl+g 搜索文件内容
```

## 插件 3: coc.nvim —— 智能补全

这是 Vim 里最接近 IDE 补全体验的插件。你在 `coc.nvim` 的支持下写代码时，输入 `console.l`，它会自动提示 `log`、`log` 的多参数版本、还有你项目里定义的相关函数。按 `Tab` 或 `Enter` 选择，自动补全。

它背后依赖的是你当前项目里使用的语言服务器。`coc.nvim` 本身是入口，它需要配合语言服务器才能提供补全能力：

- 写 Python → 需要装 `pyright` 或 `python-lsp-server`
- 写 JavaScript/TypeScript → 需要装 `typescript-language-server`
- 写 Go → 需要装 `gopls`
- 写 Java → 需要装 `jdtls`

安装方式是在 Vim 里输入 `:CocInstall coc-json coc-tsserver coc-python` 等命令。它会下载对应的语言服务器，并在你打开对应文件类型时自动加载。

```
" 3. 智能补全
Plug 'neoclide/coc.nvim', {'branch': 'release'}

" ----- coc.nvim 快捷键 -----
" 按 Tab 或 Enter 选择补全项
inoremap <silent><expr> <TAB>
    \ coc#pum#visible() ? coc#pum#next(1) : "\<Tab>"
inoremap <silent><expr> <CR>
    \ coc#pum#visible() ? coc#pum#confirm() : "\<CR>"
```

## 最终 .vimrc 中的插件配置结构：

```
" ----- 插件管理器 -----
call plug#begin('~/.vim/plugged')

Plug 'preservim/nerdtree'          " 文件树
Plug 'junegunn/fzf', { 'do': { -> fzf#install() } }
Plug 'junegunn/fzf.vim'           " 模糊搜索
Plug 'neoclide/coc.nvim', {'branch': 'release'} " 智能补全

call plug#end()

" ----- NERDTree -----
nnoremap <C-e> :NERDTreeToggle<CR>

" ----- fzf -----
nnoremap <C-p> :Files<CR>
nnoremap <C-g> :Rg<CR>

" ----- coc.nvim -----
inoremap <silent><expr> <TAB>
    \ coc#pum#visible() ? coc#pum#next(1) : "\<Tab>"
inoremap <silent><expr> <CR>
    \ coc#pum#visible() ? coc#pum#confirm() : "\<CR>"
```

## 6.4 守夜者忠告：插件是工具，不是收藏品

你可能听说过有人装了几十个插件来“武装 Vim”。但插件越多，启动越慢，配置越复杂，不同插件的快捷键相互打架的风险也越高。

插件的正确逻辑是：

**你遇到了一个“Vim 原生做不了”的需求 → 找对应的插件 → 装它 → 用起来。**

而不是：

**看到别人推荐一个插件 → 装它 → 从来不用 → 让它占据你的启动时间。**

这三款插件是“通用型”的：

- NERDTree = 你看不到文件在哪 → 解决“文件导航”需求
- fzf = 你找不到文件在哪 → 解决“文件定位”需求
- coc.nvim = 你打字太慢 → 解决“代码补全”需求

等这三款插件用顺手了，你自然会发现自己的新需求——比如“我想在 Vim 里看 Git 状态”，那时候再装一个 Git 插件也不迟。

---

## 6.5 你能拿走的（就这三件事）

1. 插件管理器用 vim-plug：轻量、三行搞定、不需要额外学习成本
2. 三款插件装完你的 Vim 就能覆盖文件树、文件搜索、代码补全三个核心需求
3. 插件不够用的时候再装，不要一次性装一堆“以后可能会用”的插件

---

## 6.6 本课作业

1. 安装 vim-plug（用终端命令或手动下载）
2. 在 `.vimrc` 里添加 `call plug#begin()` / `call plug#end()` 结构
3. 在中间添加 `Plug 'preservim/nerdtree'`
4. 保存 `.vimrc`，在 Vim 里输入 `:PlugInstall`，按回车
5. 等插件下载完成，按 `Ctrl+e` 打开文件树，浏览当前目录
6. 添加 fzf 和 coc.nvim（重复上面的步骤）
7. 尝试用 `Ctrl+p` 搜索一个文件，用 `Ctrl+g` 搜索一段内容
8. 体验一下：写完一个词之后，Vim 是否自动弹出补全提示

**守夜者宣言：** 插件是 Vim 生态里最容易被“收藏”的东西。但真正的生产力来自你每天使用的功能，不是你已经安装的插件数量。这三款插件，配上前面的 `.vimrc` 配置，足够你在终端里完成一个完整项目的编写。如果你发现它们还不够用，不是你的问题——是你已经准备好探索更多的插件世界了。到那时候，你知道怎么找，也知道怎么判断好不好用了。

---

## 选修7：多文件与多窗口 —— 告别“一个文件一个终端”

Kate 在学完前十课之后，已经能“得体”地用 Vim 编辑单个文件了。但她很快遇到了一个新问题：

“我写代码的时候，经常要同时看好几个文件——左边是 HTML 模板，右边是 CSS 样式表，偶尔还要翻一下配置文件。每次都要 `:q` 退出当前文件再打开另一个——这也太累了。Vim 能不能同时打开多个文件？”

“不但能，而且有三种方式——缓冲区、窗口、标签页。它们是 Vim 多文件管理的三件套，各有各的适用场景。”

---

## 7.1 什么是缓冲区？

缓冲区（Buffer）是 Vim 里最基本的多文件管理方式。你在 Vim 里打开一个文件，这个文件的内容就被加载到一个缓冲区里。你切换文件，就是在不同的缓冲区之间切换。缓冲区是 Vim 的核心设计——你每次 `vim 文件名` 打开一个文件，本质上就是创建了一个缓冲区。

| 命令                                        | 效果                          |
|-------------------------------------------|-----------------------------|
| <code>vim 文件1 文件2</code>                  | 启动 Vim 时同时打开多个文件            |
| <code>:e 文件名</code>                       | 在当前窗口打开另一个文件（e = edit）      |
| <code>:ls</code>                          | 列出所有已打开的缓冲区                 |
| <code>:bnext</code> （或 <code>:bn</code> ） | 切换到下一个缓冲区                   |
| <code>:bprev</code> （或 <code>:bp</code> ） | 切换到上一个缓冲区                   |
| <code>:bd</code>                          | 关闭当前缓冲区（bd = buffer delete） |

缓冲区的本质是“加载到内存中的文件”。你打开过但没有关闭的文件，仍然在缓冲区列表里，即使你当前看不到它。用 `:ls` 可以看到所有缓冲区——每个缓冲区有一个编号，前面带 `%` 的是你当前正在编辑的。

Kate 用 `vim 笔记.md 配置文件.md 样式.css` 同时打开了三个文件，然后输入 `:ls`，看到了三个缓冲区的列表。她说：“所以缓冲区就是‘文件暂存区’——我打开过的文件都在里面，随时可以切回去，不用退出再重新打开。”

## 7.2 什么是窗口？

窗口（Window）让你同时看到多个缓冲区。你可以把 Vim 的编辑区分成上下两半或左右两半，每半显示一个不同的文件。你在 02 的 VS Code 里学过分屏——`Ctrl+ $\backslash$`  把编辑区一分为二。Vim 的窗口逻辑和它一样。

| 命令                    | 效果                              |
|-----------------------|---------------------------------|
| <code>:sp 文件名</code>  | 水平分割窗口，在新窗口中打开指定文件（sp = split）  |
| <code>:vs 文件名</code>  | 垂直分割窗口，在新窗口中打开指定文件（vs = vsplit） |
| <code>Ctrl+w h</code> | 光标移到左侧窗口                        |
| <code>Ctrl+w j</code> | 光标移到下方窗口                        |
| <code>Ctrl+w k</code> | 光标移到上方窗口                        |
| <code>Ctrl+w l</code> | 光标移到右侧窗口                        |
| <code>Ctrl+w q</code> | 关闭当前窗口（q = quit）                |
| <code>Ctrl+w =</code> | 让所有窗口等大                         |

当你需要同时查看两个文件时——比如左边是 HTML 模板，右边是 Flask 视图函数——垂直分屏是最方便的。当你在一个文件里修改代码时，另一个文件的内容保持可见，你不需要在两个缓冲区之间来回切。

Kate 输入 `:vs 样式.css`，编辑区被垂直分成两半。左边是她的 HTML 文件，右边是 CSS 样式表。她说：“这就是 VS Code 的分屏。我在左边改结构，右边改样式，两边同时都在屏幕上——不用来回切了。”

“对。而且你可以继续分——左边再分一次，变成三个窗口。只要你的屏幕够大，你想同时看几个文件就看几个。”

### 7.3 什么是标签页？

标签页（Tab Page）是 Vim 里最“重”的多文件管理方式。每个标签页可以包含多个窗口，每个窗口显示一个缓冲区。标签页和缓冲区的区别在于：缓冲区是文件的内存副本，标签页是窗口的容器。你关闭一个标签页，里面的所有窗口都会关闭，但缓冲区还在内存里——除非你显式关闭缓冲区。

| 命令                                             | 效果           |
|------------------------------------------------|--------------|
| <code>:tabnew 文件名</code>                       | 新建标签页并打开指定文件 |
| <code>:tabclose</code> （或 <code>:tabc</code> ） | 关闭当前标签页      |
| <code>gt</code>                                | 切换到下一个标签页    |
| <code>gT</code>                                | 切换到上一个标签页    |

标签页适合不同的“工作场景”——比如一个标签页放前端代码（HTML + CSS），另一个标签页放后端代码（Flask 视图），再开一个标签页写笔记。每个标签页是一个独立的工作区，各自有各自的窗口布局。切换标签页比切换窗口更快——`gt` 一下就到另一个场景了。

Kate 试了 `:tabnew README.md`，编辑区上方出现了一个标签栏，她可以在几个标签页之间用 `gt` 快速切换。她说：“所以标签页就是‘工作区隔离’——一个工作区写前端，一个工作区写后端，一个工作区写笔记。比窗口更重，但每个场景互不干扰。”

### 7.4 缓冲区、窗口、标签页——三者的关系

这三者的关系可以用一句话总结：**缓冲区是文件，窗口是看文件的镜头，标签页是装镜头的箱子。**一个文件只能加载到一个缓冲区里，但你可以在多个窗口里同时看同一个缓冲区——就像你用多个镜头从不同角度拍同一个东西。你切换标签页，镜头和箱子一起换——原来拍的是前端，现在拍的是后端。

| 概念  | 你可以有 | 关闭了会怎样            |
|-----|------|-------------------|
| 缓冲区 | 多个   | 文件还在硬盘上，缓冲区被释放    |
| 窗口  | 多个   | 当前窗口关闭，其他窗口还在     |
| 标签页 | 多个   | 标签页内所有窗口关闭，但缓冲区还在 |

## 7.5 课后作业

你正在写今天的学习笔记，需要同时参考《守夜者之书》的 Markdown 语法速查表和之前记的渗透测试概念。你可以用下面这套配置来管理它们：

1. 打开你今天要写的笔记：在终端里输入 `vim 今天的学习笔记.md`
2. 垂直分屏，右侧固定显示渗透测试概念：输入 `:vs 概念.md`
3. 新建标签页，打开 Markdown 语法速查表：输入 `:tabnew 速查表.md`

现在你有两个标签页——第一个标签页里是“笔记+概念参考”的左右分屏布局，第二个标签页里是语法速查表。你可以用 `gt` 在两者之间秒切，用 `Ctrl+w h` 和 `Ctrl+w l` 在分屏窗口之间左右移动。用 `:ls` 随时查看所有缓冲区——三个文件全部在内存里，不需要重新打开。

*Kate 按照这套配置搭好了她的写作环境，说：“我现在左边写笔记，右边固定显示渗透测试概念——写到忘记某个术语的时候，眼睛一扫右边就想起来了。gt 一键切到另一个标签页查 Markdown 语法。我手指不用离开键盘去摸鼠标，所有文件全在 Vim 里。”*

## 7.6 本节小结

| 概念  | 一句话解释          | 什么时候用            |
|-----|----------------|------------------|
| 缓冲区 | 加载到内存中的文件      | 打开多个文件，在它们之间快速切换 |
| 窗口  | 把编辑区分成多个区域     | 同时查看两个或更多文件      |
| 标签页 | 窗口的容器，独立的“工作区” | 为不同的工作场景分别设置窗口布局 |

《守夜者之书 · 03. Vim 从入门到得体》 完

# 附录

## 附录 A：vi 与 vim 的完整对比表

这一节写给两种人：

- 好奇“vi 和 vim 到底差在哪”的读者
- 将来可能在服务器上只遇到 `vi`、需要知道“哪些功能不能用”的守夜者

如果你只是想安心用 Vim 写代码、改配置，这一节可以跳过。

如果你想知道“Vim 到底比 Vi 强在哪”——往下看。

| 功能            | vi          | vim        | 说明                                                            |
|---------------|-------------|------------|---------------------------------------------------------------|
| 多级撤销          | ✗<br>只能撤销一次 | ✓<br>无限撤销树 | vi 里按 <code>u</code> 只能撤一步，再按是重做；vim 里可以一直撤                   |
| 语法高亮          | ✗           | ✓          | vi 的世界是黑白的，vim 能给代码上色                                         |
| 可视模式          | ✗           | ✓          | vi 不能 <code>v</code> 选文字再操作，vim 可以                            |
| 图形界面版本        | ✗           | ✓ (gVim)   | vi 只有终端版，vim 有独立窗口版本                                          |
| 插件系统          | ✗           | ✓          | vi 是死的，vim 是活的                                                |
| 多窗口分屏         | ✗           | ✓          | vi 一次只能看一个文件，vim 可以 <code>:split</code> 左右/上下分屏               |
| 远程文件编辑        | ✗           | ✓          | vim 可以直接编辑 <code>scp://</code> 或 <code>ftp://</code> 文件，vi 不行 |
| 中文帮助文档        | ✗           | ✓ (需安装)    | vi 只有英文，vim 的中文帮助可以单独安装                                       |
| 撤销历史持久化       | ✗           | ✓          | vim 可以把撤销历史存到文件里，重启后还能 <code>u</code> 回去                      |
| 鼠标支持          | ✗           | ✓          | 在终端里 vim 可以用鼠标点击定位、选择，vi 不行                                   |
| 自动补全          | ✗           | ✓          | vim 有 <code>Ctrl+n</code> / <code>Ctrl+p</code> 单词补全，vi 没有    |
| 加密支持          | ✗           | ✓          | vim 可以用 <code>-x</code> 参数加密文件，vi 不支持                         |
| 宏录制<br>(含寄存器) | ✗           | ✓          | vi 完全没有 <code>q</code> 录制宏的能力，vim 有完整的宏系统                     |

## 一句话总结

vi 是 Vim 的“半成品祖先”。它在 1976 年能做的事情，在今天依然能做——但如果你已经习惯了语法高亮、无限撤销、可视模式、多窗口、宏录制、自动补全，vi 会让你觉得“缺了太多东西”。

绝大多数 Linux 发行版默认安装的是 **vim-tiny** 或 **vim-minimal**，它们在功能上接近 vi。完整版的 Vim 需要额外安装。所以如果有一天你 SSH 进一台服务器，输入 `vim` 发现语法高亮没了、`u` 只能撤一步——不是你不会用，是这台机器只装了 vi 的兼容版本，不是完整的 Vim。

那时候，你有两个选择：

1. 用你学会的 Vim 知识，在有限的 vi 子集里完成必要的配置修改
2. 输入 `sudo apt install vim`（或 `yum install vim`），把完整的 Vim 请进门



# 附录 B：常用命令速查卡

这一页是给你放在手边随时翻的。  
不需要背——等你用熟了，你会发现你根本不需要翻开它。  
但在那之前，它是你在 Vim 里迷路时的“路标”。

## 移动光标

| 命令                         | 效果            |
|----------------------------|---------------|
| <code>h / j / k / l</code> | 左 / 下 / 上 / 右 |
| <code>w / b</code>         | 下一个词 / 上一个词   |
| <code>0</code>             | 行首（数字零）       |
| <code>\$</code>            | 行尾            |
| <code>gg</code>            | 文件首           |
| <code>G</code>             | 文件尾           |
| <code>Ctrl + f</code>      | 下一页（forward）  |
| <code>Ctrl + b</code>      | 上一页（backward） |

## 编辑文本

| 命令               | 效果      |
|------------------|---------|
| <code>i</code>   | 光标前插入   |
| <code>a</code>   | 光标后插入   |
| <code>I</code>   | 行首插入    |
| <code>A</code>   | 行尾插入    |
| <code>o</code>   | 下一行插入   |
| <code>O</code>   | 上一行插入   |
| <code>x</code>   | 删除光标下字符 |
| <code>dd</code>  | 删除整行    |
| <code>dw</code>  | 删除一个词   |
| <code>d\$</code> | 删除到行尾   |
| <code>yy</code>  | 复制整行    |

| 命令       | 效果      |
|----------|---------|
| yw       | 复制一个词   |
| y\$      | 复制到行尾   |
| p        | 光标后粘贴   |
| P        | 光标前粘贴   |
| u        | 撤销      |
| Ctrl + r | 重做      |
| .        | 重复上一次修改 |

文件操作

| 命令     | 效果                 |
|--------|--------------------|
| :w     | 保存                 |
| :q     | 退出                 |
| :wq    | 保存并退出              |
| :x     | 保存并退出（仅当文件有修改时才写盘） |
| ZZ     | 保存并退出（等效 :x ）      |
| :q!    | 不保存强制退出            |
| ZQ     | 不保存强制退出（等效 :q! ）   |
| :e 文件名 | 打开另一个文件            |

搜索与替换

| 命令         | 效果         |
|------------|------------|
| /关键词       | 向下搜索       |
| ?关键词       | 向上搜索       |
| n          | 下一个匹配      |
| N          | 上一个匹配      |
| :%s/旧/新/g  | 全文替换       |
| :%s/旧/新/gc | 全文替换（逐个确认） |

可视模式

| 命令       | 效果          |
|----------|-------------|
| v        | 进入可视模式（字符级） |
| V        | 进入可视模式（行级）  |
| Ctrl + v | 进入可视模式（块级）  |

文本对象（选修）

| 命令        | 效果        |
|-----------|-----------|
| ci"       | 修改双引号内的内容 |
| ci'       | 修改单引号内的内容 |
| ci( / ci) | 修改括号内的内容  |
| ci{ / ci} | 修改花括号内的内容 |
| dip       | 删除当前段落    |
| vip       | 选中当前段落    |

命令组合公式

动词 + 名词 = Vim 命令

| 动词 | 含义            |
|----|---------------|
| d  | 删除            |
| y  | 复制            |
| c  | 修改（删除后进入插入模式） |
| >  | 右缩进           |
| <  | 左缩进           |

| 名词 | 含义    |
|----|-------|
| w  | 到下一个词 |
| \$ | 到行尾   |

| 名词 | 含义     |
|----|--------|
| G  | 到文件末尾  |
| i" | 引号内的内容 |

**守夜者提醒：**  
这张速查卡不是“考试范围”。  
它是你迷路时的路标——等你走熟了，你就不需要低头看路了。

## 附录 C：推荐阅读 —— 《Vim 实用技巧（第 2 版）》

恭喜你。你已经走到这里了。

你翻开了这本《十节课极简入门 Vim》，读完了正文，走到了附录。  
你现在知道 `hjk1` 是什么意思、什么时候按 `ESC`、怎么用 `ci"` 改引号里的内容。  
你甚至知道 `.` 命令和宏的区别。  
更重要的是——你开始理解 Vim 不是一堆快捷键，而是一套语法。

你已经是一个“能活下来”的 Vim 用户了。

但现在的问题是：你站在“会用”和“熟练”之间。你知道语法，但你的手指还不够快。你知道文本对象的概念，但在真实项目中还不能自然地用出来。你想看到“高手是怎么思考和操作的”——不只是命令，而是思维过程。  
这时候，你需要翻开那本在 Vim 社区里被反复推荐的书：

### 《Vim 实用技巧（第 2 版）》

- 作者：Drew Neil
- 豆瓣评分：9.2
- 英文原版：《Practical Vim, Second Edition》
- 定位：不是命令大全，是“Vim 心法”

为什么是这本书，而不是别的？

| 其他 Vim 书籍 | 《Vim 实用技巧》                |
|-----------|---------------------------|
| 按命令分类列举   | 按“场景”组织——你想解决什么问题，翻到对应的技巧 |
| 告诉你“按什么键” | 告诉你“为什么会想到按这个键”           |
| 适合当字典查    | 适合当武功秘籍练                  |
| 读完知道了更多命令 | 读完知道了“怎么思考编辑任务”           |

Drew Neil 是一位资深 Vim 用户和培训师。他写的这本书不是教科书，而是一本“高手在工作现场的操作实录”。他先描述一个编辑任务（比如“重构函数名”），然后带你走一遍他是怎么想的、按了什么键、为什么这么按——你看

到的不是命令列表，是思维链条。

怎么读这本书：

- 1. 先通读一遍。遇到感兴趣的部分直接在你的 Vim 里练一遍
- 2. 不要一口气读完。每天读 5-10 个技巧，每个技巧在 Vim 里敲一遍
- 3. 不要只读“技巧”，要读“为什么”——作者在每个技巧里都会解释他的思考过程，那才是这本书真正有价值的部分
- 4. 读完后，把你最常用的 5 个技巧记下来，贴在桌面上，用一个月

这本书里的很多技巧你会在读的时候觉得“我知道这个命令”——但重点不是“你知道”，而是“你在工作中会不会想到用它”。Drew Neil 教你的是后者。

读完这本教程 + 这本书之后，你站在哪里？

| 阶段           | 状态                                              |
|--------------|-------------------------------------------------|
| 读这本教程前       | 不敢用 Vim，进了不知道怎么退出                               |
| 读完这本教程       | 能写、能改、能保存、能退出、能用常见命令，能舒服地配置 <code>.vimrc</code> |
| 读完《Vim 实用技巧》 | 能在工作中自然使用 Vim，看到编辑任务时自动想“用 Vim 怎么做会更高效”         |

“能活下来”和“能打得顺手”之间，就是这本书的距离。

**守夜者提醒：**  
不要把这本书当字典查。它的编排方式是“技巧”，不是“索引”。  
如果你只是想找一个命令，用 `:help` 更快。  
但如果你想理解“Vim 高手是怎么思考的”——这就是那本书。  
你不需要读完才用。**这本书是给“在用 Vim”的人写的。** 你已经在这条路上了。

附录 D：常见坑与解法（新手救急）

这一页是你在 Vim 里“翻车”时用的。  
如果你卡住了，翻到这里，找到你的现象，照着做。  
大部分“Vim 坏了”的错觉，其实只是你按错了一个键。

| 现象                        | 原因      | 解法                                          |
|---------------------------|---------|---------------------------------------------|
| 按 <code>i</code> 没反应，打不出字 | 你不在普通模式 | 按 <code>Esc</code> 回到普通模式，再按 <code>i</code> |

| 现象                                                                    | 原因                                               | 解法                                                                                               |
|-----------------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 按 <code>h j k l</code> 变成了输入字母                                        | 你在插入模式                                           | 按 <code>Esc</code> 退出插入模式再试                                                                      |
| <code>:wq</code> 报错<br><code>E212: Can't open file for writing</code> | 没有写权限                                            | <code>:w !sudo tee %</code> (临时用 <code>sudo</code> 写入当前文件)                                       |
| 打开文件全是乱码                                                              | 文件编码不是 UTF-8                                     | <code>:e ++enc=utf-8</code> 用 UTF-8 重新加载当前文件                                                     |
| 按 <code>Ctrl+s</code> 卡死了                                             | 你按了 XOFF 流控制信号, 终端暂停输出                           | <code>Ctrl+q</code> 解除暂停                                                                         |
| 不小心按了 <code>Ctrl+z</code> , Vim 不见了                                   | Vim 被挂起到后台                                       | 在终端里输入 <code>fg</code> , 按回车, 回到 Vim                                                             |
| 撤销撤多了, 想“反撤销”                                                         | 不知道 <code>Ctrl+r</code>                          | <code>Ctrl+r</code> 重做 (撤销你的撤销)                                                                  |
| 按 <code>dd</code> 删了一行, 想恢复                                           | 不知道 <code>u</code>                               | <code>u</code> 撤销                                                                                |
| 按 <code>:</code> 想输命令, 结果啥也没发生                                        | 你在插入模式, <code>:</code> 被当成了普通字符                  | 按 <code>Esc</code> 退出后再按 <code>:</code>                                                          |
| 按 <code>p</code> 粘贴, 出来的内容不对                                          | 你复制的内容里含有特殊字符或格式                                 | 先 <code>:set paste</code> 再进入插入模式粘贴                                                              |
| 搜索高亮一直不消失                                                             | 搜索匹配项高亮持续保留                                      | <code>:nohlsearch</code> (或简写 <code>:noh</code> ) 取消高亮                                           |
| 滚动太快, 滚过了                                                             | <code>Ctrl+f</code> / <code>Ctrl+b</code> 翻页幅度太大 | 用 <code>Ctrl+d</code> (向下半页) 和 <code>Ctrl+u</code> (向上半页) 细调                                     |
| 按 <code>u</code> 撤销后, 按 <code>.</code> 想重复上次操作, 结果没反应                 | <code>.</code> 不重复撤销操作                           | <code>.</code> 只重复编辑操作, 不重复 <code>u</code> 。想重做用 <code>Ctrl+r</code>                             |
| 不小心删了一行, 但 <code>u</code> 没恢复                                         | 你可能在宏录制或可视模式里操作过                                 | 退出当前模式, 再按 <code>u</code> 。如果还不行, 按 <code>U</code> 恢复整行最初状态                                      |
| 在终端里 Vim 的颜色很奇怪                                                       | 终端主题和 Vim 配色冲突                                   | 检查 <code>.vimrc</code> 里的 <code>colorscheme</code> 是否支持当前终端, 或加一行 <code>set termguicolors</code> |

### 三句救命口诀

记不住上面这张表的时候, 记住这三句话就够了:

- 卡住了按 `Esc` ——你永远不知道自己在什么模式, 但 `Esc` 能让你回普通模式。按它, 然后重新开始。

2. **想退出** `:q` ——不让退就加 `!`：`:q!`。它能让你从任何卡死状态离开 Vim。
3. **做错了按** `u` ——除了退出，Vim 里最不容易出错的就是撤销。你随时可以 `u` 回去。

---

## 终端卡死 & 恢复（补充说明）

你可能会遇到一种情况：在终端里用 Vim 时，你按了 `Ctrl+s`（或误触了），屏幕突然卡住，光标不再响应，连 `Esc` 都没反应。这不是 Vim 的问题——这是你触发了终端的\*\*流控制（XON/XOFF）\*\*机制，终端本身暂停了屏幕输出。

**症状：** 屏幕冻结，按任何键都没反应，但你敲的内容可能在后台排队。你并没有失去控制权——只是屏幕显示被暂停了。

**解法：** 按 `Ctrl+q` 恢复屏幕输出。如果你按了多次 `Ctrl+s`，多按几次 `Ctrl+q` 也能全部解除。

如果你真的遇到了无法恢复的情况，不要直接关闭终端窗口——那样会丢失你未保存的内容。正确的做法是：

1. 先尝试 `Ctrl+q` 解除屏幕冻结
2. 如果无效，按 `Ctrl+z` 把 Vim 挂起到后台，输入 `fg` 恢复
3. 如果仍然无效，按 `Ctrl+`（某些终端）强制中断

如果你没有保存的内容，不要直接关闭终端。

---

## 写在最后

**读完这本教程，你不是 Vim 高手。但你已经能得体地用它写代码、改配置、活着退出来。**

这已经超越了 80% 的 Vim 初学者。

想成为剩下 20%？附录 C 那本书，是你的下一站。

不想成为高手？现在的水平，足够你用 Vim 做大部分事情了。工具是为你服务的，不是让你为工具服务的。

**记住：Vim 的至高境界，不是会按多少命令，而是手指已经忘了命令，脑子在想，代码已经出来了。**

今夜如此，夜夜皆然。

—— 守夜者